基于 Chisel 的数字设计

基于 Chisel 的数字设计

Martin Schoeberl

中文版

Copyright © 2016–2019 Martin Schoeberl



本作品采用 Creative Commons Attribution-ShareAlike 4.0 国际许可协议（CC BY-SA 4.0）授权。http://creativecommons.org/licenses/by-sa/4.0/

英文原版书名：*Digital Design with Chisel*
作者邮箱：martin@jopdesign.com
原书源码：https://github.com/schoeberl/chisel-book

2019 年由 Kindle Direct Publishing 出版，
https://kdp.amazon.com/

中文版说明

本中文译本依据原书英文版翻译，同样以 CC BY-SA 4.0 协议发布。译文中保留了代码、命令、文件名与程序标识符的英文原文，以便读者直接运行示例。翻译规范见项目根目录的 `TRANSLATION_GUIDE.md`。

ISBN 9781689336031（英文原版）

英文原版由 Martin Schoeberl 排版；中文版使用 XeLaTeX 排版。

目录

译者说明 viii

第一章 前言 1

第二章 引言 2

2.1 安装 Chisel 与 FPGA 工具 . . . 3
2.2 Hello World . . . 3
2.3 Chisel 的 Hello World . . . 4
2.4 为 Chisel 配置 IDE . . . 4
2.5 源码获取与电子书特性 . . . 5
2.6 延伸阅读 . . . 5
2.7 练习 . . . 6

第三章 基本组件 8

3.1 信号类型与常量 . . . 8
3.2 组合电路 . . . 9
3.2.1 多路选择器 . . . 11
3.3 寄存器 . . . 12
3.3.1 计数 . . . 12
3.4 使用 Bundle 和 Vec 组织信号 . . . 12
3.5 Chisel 生成硬件 . . . 14
3.6 练习 . . . 15

第四章 构建流程与测试 16

4.1 使用 sbt 构建项目 . . . 16
4.1.1 源码组织 . . . 16
4.1.2 运行 sbt . . . 18
4.2 使用 Chisel 进行测试 . . . 18
4.2.1 使用 ScalaTest . . . 20
4.3 练习 . . . 21
4.3.1 一个最小项目 . . . 21
4.3.2 测试练习 . . . 23

第五章 组件 24

5.1 Chisel 中的组件就是模块 . . . 24
5.2 算术逻辑单元 . . . 27
5.3 批量连接 . . . 28

5.4 用函数实现轻量级组件 29

第六章 组合逻辑构建模块 30

6.1 组合电路 30
6.2 译码器 31
6.3 编码器 33
6.4 练习 34

第七章 时序逻辑构建模块 35

7.1 寄存器 35
7.2 计数器 38
7.2.1 向上计数与向下计数 39
7.2.2 用计数器生成定时 40
7.2.3 极客计数器 41
7.3 存储器 41
7.4 练习 44

第八章 有限状态机 45

8.1 基本的有限状态机 45
8.2 用米利型 FSM 加快输出 48
8.3 摩尔型与米利型对比 51
8.4 练习 53

第九章 相互通信的状态机 54

9.1 灯光闪烁器示例 54
9.2 带数据通路的状态机 57
9.2.1 位计数示例 58
9.3 Ready-Valid 接口 61

第十章 硬件生成器 63

10.1 使用参数进行配置 63
10.1.1 简单参数 63
10.1.2 带类型参数的函数 63
10.1.3 带类型参数的模块 65
10.1.4 参数化 Bundle 65
10.2 生成组合逻辑 66
10.3 使用继承 68

第十一章 设计实例 **71**
11.1 FIFO 缓冲区 71
11.2 串口 73
11.3 FIFO 设计的变体 78
11.4 练习 79
11.4.1 探索 FIFO 变体 79
11.4.2 UART 79

第十二章 处理器设计 **81**
12.1 从 ALU 开始 81
12.2 指令译码 84
12.3 汇编指令 86
12.4 练习 87

第十三章 为 Chisel 做贡献 **89**
13.1 搭建开发环境 89
13.2 测试 90
13.3 通过 Pull Request 贡献代码 90
13.4 练习 90

第十四章 总结 **91**

附录 A Chisel 项目 **92**

附录 B Chisel 2 **93**

插图目录

3.1 表达式 `(a & b) | c` 对应的逻辑。连线可以是单个位，也可以是多个位。Chisel 表达式与原理图表达的是同一个电路。 9
3.2 一个基本的 2:1 多路选择器。 11

4.1 Chisel 项目（使用 `sbt`）的源码树 16

5.1 由层次化组件构成的设计。 24
5.2 算术逻辑单元，简称 ALU。 27

6.1 2 位到 4 位译码器。 32
6.2 4 位到 2 位编码器。 33

7.1 基于 D 触发器的寄存器。 35
7.2 带同步复位的 D 触发器寄存器。 36
7.3 带复位寄存器的波形图。 37
7.4 带使能信号的 D 触发器寄存器。 37
7.5 带使能信号寄存器的波形图。 38
7.6 加法器与寄存器构成计数器。 38
7.7 生成慢频率节拍的波形图。 40
7.8 同步存储器。 42
7.9 带前递电路的同步存储器，具有确定的写后读行为。 44

8.1 有限状态机（摩尔型）。 45
8.2 告警 FSM 的状态图。 46
8.3 上升沿检测器（米利型 FSM）。 49
8.4 米利型有限状态机。 49
8.5 上升沿检测器作为米利型 FSM 的状态图。 49
8.6 上升沿检测器作为摩尔型 FSM 的状态图。 51
8.7 上升沿检测的米利型和摩尔型 FSM 波形。 52

9.1 拆分为主 FSM 和定时器 FSM 的灯光闪烁器。 54
9.2 拆分为主 FSM、定时器 FSM 和计数器 FSM 的灯光闪烁器。 56
9.3 带数据通路的状态机。 58
9.4 位计数 FSM 的状态图。 59
9.5 位计数电路的数据通路。 60
9.6 ready-valid 流控。 62

11.1 写入方、FIFO 缓冲区和读取方。 71
11.2 UART 传输一个字节。 73

表格目录

3.1 Chisel 定义的硬件运算符。 10
3.2 Chisel 定义的硬件函数，作用于 v。 11

6.1 2 到 4 译码器的真值表。 32
6.2 4 到 2 编码器的真值表。 33

8.1 告警 FSM 的状态表。 46

12.1 Leros 指令集。 82

代码清单目录

2.1 用 Chisel 实现的硬件 Hello World 4
7.1 1 KB 的同步存储器。 42
7.2 带前递电路的存储器。 43
8.1 告警 FSM 的 Chisel 代码。 46
8.2 用米利型 FSM 检测上升沿。 50
8.3 用摩尔型 FSM 检测上升沿。 51
9.1 灯光闪烁器的主 FSM。 55
9.2 两次重构后的灯光闪烁器主 FSM。 57
9.3 位计数电路的顶层。 58
9.4 位计数电路的数据通路。 60
9.5 位计数电路的 FSM。 61
10.1 读取文本文件以生成逻辑表。 66
10.2 二进制到二进制编码十进制（BCD）的转换。 67
10.3 用计数器生成节拍。 68
10.4 用于测试不同版本 ticker 的测试器。 68
10.5 用递减计数器生成节拍。 69
10.6 通过递减到 -1 生成节拍。 69
10.7 ticker 测试的 ScalaTest 测试规范。 70
11.1 气泡 FIFO 的单级缓冲区。 72
11.2 由一组 FIFO 气泡级组成的 FIFO。 72
11.3 串口发送器。 74
11.4 带 ready/valid 接口的单字节缓冲区。 75
11.5 带额外缓冲区的发送器。 75
11.6 串口接收器。 76
11.7 通过串口发送“Hello World!”。 77
11.8 在串口上回显数据。 77
12.1 Leros 的 ALU。 81
12.2 用 Scala 编写的 Leros ALU 功能。 83
12.3 Leros 汇编器的主要部分。 87

译者说明

本书是 Martin Schoeberl 所著 *Digital Design with Chisel* 的中文译本。原书以简洁的方式介绍如何使用硬件构建语言 Chisel 进行数字系统设计，内容涵盖基本组合电路与时序电路、构建与测试流程、有限状态机、硬件生成器，以及一个完整的处理器设计实例。

翻译工作遵循项目根目录下 `TRANSLATION_GUIDE.md` 中的规范，主要原则如下：

- 忠实传达原文的技术含义，不增删事实、条件与限制；
- 全书术语统一，关键术语首次出现时附英文原文；
- 代码、命令、文件名、信号名与程序标识符保持英文原样，以保证示例可直接编译运行，并与上游源码一致；
- 正文采用中文全角标点，中英文之间保留空格，力求通顺、简洁，避免翻译腔。

原书中的插图由作者使用英文标注绘制，图内文字未作改动；正文与图注中的相应术语已译为中文，读者可对照阅读。译文中的错误与不当之处由译者负责，欢迎通过项目仓库提交 issue 或 pull request。

译者

2026 年

第一章　前言

本书是一本数字设计导论，重点介绍如何使用硬件构建语言 Chisel。Chisel 将软件工程领域的进展——例如面向对象语言和函数式语言——引入数字设计。

本书面向硬件设计师和软件工程师。熟悉 Verilog 或 VHDL 的硬件设计师，可以在下一个 ASIC 或 FPGA 设计中改用这种现代语言，从而提高工作效率。了解面向对象编程和函数式编程的软件工程师，则可以利用已有知识来编写硬件，例如在云端运行的 FPGA 加速器。

本书采用的方法是通过小型到中型的典型硬件组件，来探索如何使用 Chisel 进行数字设计。

致谢

我要感谢所有为 Chisel 做出贡献的人，他们创造了如此出色的硬件构建语言。使用 Chisel 令人愉悦，因此值得为它写一本书。我感谢整个 Chisel 社区：他们热情友好，总是不厌其烦地回答关于 Chisel 的问题。

我也要感谢近年来选修高级计算机体系结构课程的学生，他们中的大多数在期末项目中选择了 Chisel。感谢你们走出舒适区，踏上学习和使用一种前沿硬件描述语言的旅程。你们提出的许多问题帮助塑造了这本书。

第二章　引言

本书介绍如何使用现代硬件构建语言 Chisel [2] 进行数字系统设计。与常见的数字设计教材相比，本书关注更高的抽象层次，使读者能够在更短的时间内构建更复杂、相互协作的数字系统。

本书和 Chisel 面向两类开发者：(1) 硬件设计师；(2) 软件程序员。熟悉 VHDL 或 Verilog、并使用 Python、Java 或 Tcl 等其他语言生成硬件的设计师，可以转向一种统一的硬件构建语言，让硬件生成成为语言本身的一部分；而对硬件设计感兴趣的程序员也可以借此进入这一领域，例如，未来 Intel 的芯片将包含可编程硬件以加速程序执行。完全可以把 Chisel 作为你的第一门硬件描述语言。

Chisel 将软件工程的进展，例如面向对象语言和函数式语言，引入数字设计。Chisel 不仅能在寄存器传输级描述硬件，还允许你编写硬件生成器。

如今，硬件通常使用硬件描述语言来描述。即使借助 CAD 工具，手绘硬件组件的时代也已经结束。一些高层次原理图可以给出系统的概览，但并不适合用来描述系统本身。最常见的两种硬件描述语言是 Verilog 和 VHDL。两者都历史悠久，背负着许多历史包袱，而且语言中哪些结构可以综合为硬件，其边界也在不断变化。请不要误解：VHDL 和 Verilog 完全能够描述可综合为 ASIC 的硬件模块。在使用 Chisel 进行硬件设计时，Verilog 充当测试与综合的中间语言。

本书不是硬件设计及其基础的通用入门教材。如果你想了解数字设计的基础知识，例如如何用 CMOS 晶体管搭建逻辑门，请参阅其他数字设计教材。本书希望在一个符合当前实践的抽象层次上讲授数字设计，用于描述 ASIC 或以 FPGA 为目标的设计。[1] 本书假定读者具备布尔代数和二进制数制的基础知识。此外，还假定读者具备某种编程语言的编程经验。不需要了解 Verilog 或 VHDL。Chisel 完全可以成为你描述数字硬件的第一门编程语言。由于示例的构建过程基于 `sbt` 和 `make`，掌握命令行界面（CLI，也称为终端或 Unix shell）的基本知识会有所帮助。

Chisel 本身并不是一门庞大的语言。其基本结构可以放在一页纸上，几天之内就能学会。因此，本书也不算厚重。Chisel 无疑比承载诸多历史包袱的 VHDL 和 Verilog 更小。Chisel 的强大之处在于它嵌入在 Scala 之中，而 Scala 本身就是一门富有表现力的语言。Chisel 继承了 Scala 的这一特点：它是一门“会逐渐让你喜欢上的语言” [12]。不过，Scala 并不是本书的主题。Odersky 等人的教材 [12] 对 Scala 做了通用介绍。本书是数字设计与 Chisel 语言的教程，而不是 Chisel 语言参考手册，也不是一本完整的芯片设计书籍。

本书展示的所有代码示例都摘自经过编译和测试的完整程序，因此这些代码不应包含任何语法错误。代码示例可以从本书的 GitHub 仓库获取。除了展示 Chisel 代码，我们还尽量呈现有用的设计以及良好硬件描述风格的原则。

本书针对在笔记本电脑或平板电脑（例如 iPad）上阅读进行了优化。正文中包含了延伸阅

[1] 由于作者对 FPGA 比 ASIC 更为熟悉，书中展示的某些设计优化针对的是 FPGA 技术。

读的链接，大多指向 Wikipedia 条目。

2.1 安装 Chisel 与 FPGA 工具

Chisel 是一个 Scala 库，安装 Chisel 和 Scala 最简单的方法是使用 Scala 构建工具 `sbt`。Scala 本身依赖于 Java JDK 1.8 的安装。

在 Mac OS X 上，使用包管理器 Homebrew 安装 `sbt`：

```
$ brew install sbt
```

在基于 Debian 的 Ubuntu 上，程序通常从 Debian 软件包（.deb）安装。但截至本书写作时，`sbt` 还没有可直接安装的软件包，因此安装过程稍显繁琐：

```
echo "deb https://dl.bintray.com/sbt/debian /" | \
  sudo tee -a /etc/apt/sources.list.d/sbt.list
sudo apt-key adv --keyserver hkp://keyserver.ubuntu.com:80 \
  --recv 2EE0EA64E40A89B84B2DF73499E82A75642AC823
sudo apt-get update
sudo apt-get install sbt
```

Chisel 和 Scala 也可以在 Windows 下安装和使用。`sbt` 可以通过 Windows 安装程序安装，参见：在 Windows 上安装 sbt。

要为 FPGA 构建硬件，还需要综合工具。两大 FPGA 厂商 Intel[2] 和 Xilinx 都提供免费工具版本，覆盖小型到中型 FPGA。这些中型 FPGA 足以构建多核 RISC 风格的处理器。Intel 提供 Quartus Prime Lite Edition，Xilinx 提供 Vivado Design Suite, WebPACK Edition。

2.2 Hello World

每本介绍编程语言的书都应从一个最小示例开始，也就是所谓的 *Hello World* 示例。下面是第一种写法：

```
object HelloScala extends App{
  println("Hello Chisel World!")
}
```

用 `sbt` 编译并运行这个简短的程序：

```
$ sbt "runMain HelloScala"
```

[2]原 Altera

将得到 Hello World 程序预期的输出：

```
[info] Running HelloScala
Hello Chisel World!
```

不过，这是 Chisel 吗？这段代码会生成硬件来打印字符串吗？不是。这只是普通的 Scala 代码，并不是硬件设计中有代表性的 Hello World 程序。

2.3 Chisel 的 Hello World

那么，硬件设计中的 Hello World 程序对应什么？什么是最小、有用且可见的设计？闪烁的 LED 就是硬件（乃至嵌入式软件）版本的 Hello World。只要 LED 能闪烁，我们就准备好去解决更大的问题了！

```
class Hello extends Module {
  val io = IO(new Bundle {
    val led = Output(UInt(1.W))
  })
  val CNT_MAX = (50000000 / 2 - 1).U;

  val cntReg = RegInit(0.U(32.W))
  val blkReg = RegInit(0.U(1.W))

  cntReg := cntReg + 1.U
  when(cntReg === CNT_MAX) {
    cntReg := 0.U
    blkReg := ~blkReg
  }
  io.led := blkReg
}
```

代码清单 2.1： 用 Chisel 实现的硬件 Hello World

代码清单 2.1 展示了一个用 Chisel 描述的闪烁 LED。不必理解这段代码的所有细节，后续章节会逐一介绍。这里只需注意：电路通常由高频时钟驱动，例如 50 MHz，因此需要一个计数器来产生以 Hz 为单位的定时，才能看到肉眼可见的闪烁。在上面的例子中，计数器从 0 计数到 25000000-1，然后翻转闪烁信号（`blkReg := ~blkReg`）并重新开始计数（`cntReg := 0.U`）。这样，硬件就会以 1 Hz 的频率让 LED 闪烁。

2.4 为 Chisel 配置 IDE

本书不对你的编程环境或编辑器作任何假设。仅使用命令行中的 `sbt` 和你喜欢的编辑器，就足以轻松学习基础知识。按照其他书的惯例，需要在 shell、终端或 CLI 中输入的命令前面都有一个 `$` 字符，这个字符不需要输入。例如，下面是 Unix 的 `ls` 命令，用于列出当前目录中

的文件:

```
$ ls
```

话虽如此，集成开发环境（IDE）可以在后台运行编译器，从而加快编码速度。由于 Chisel 是一个 Scala 库，所有支持 Scala 的 IDE 也适合用于 Chisel。在 Eclipse 和 IntelliJ 中，都可以根据 `build.sbt` 中的 sbt 项目配置生成项目。可以通过以下命令创建 Eclipse 项目:

```
$ sbt eclipse
```

然后将该项目导入 Eclipse。[3] 在 IntelliJ 中，可以从现有源码创建新项目，然后选择 `build.sbt`。

Visual Studio Code 搭配 Scala Metals，除了安装 Visual Studio Code 和 Scala Metals 插件之外无需其他配置。Scala Metals 能够自动完成各种设置，包括但不限于代码补全、下载 Chisel 源码以及基于 `build.sbt` 构建项目。

2.5 源码获取与电子书特性

本书是开源的，托管在 GitHub 上：chisel-book。书中展示的所有 Chisel 代码示例都包含在该仓库中。这些代码可以用较新版本的 Chisel 编译，许多示例还附带了测试平台。我们还在配套仓库 chisel-examples 中收集了更大的 Chisel 示例。如果你发现书中的错误或笔误，通过 GitHub pull request 提交修改是最方便的改进方式。你也可以在 GitHub 上提交 issue，或者直接发送一封传统的电子邮件来提供反馈或改进建议。

本书以 PDF 电子书和传统纸质书两种形式免费提供。电子书版本包含指向更多资源和 Wikipedia 条目的链接。对于无法直接纳入本书的背景知识（例如二进制数制），我们使用 Wikipedia 条目进行补充。电子书的版式针对平板电脑（例如 iPad）上的阅读进行了优化。

2.6 延伸阅读

下面是数字设计与 Chisel 的延伸阅读列表:

- Digital Design: A Systems Approach，作者 William J. Dally 和 R. Curtis Harting，是一本现代数字设计教材。该书有两个版本：分别使用 Verilog 或 VHDL 作为硬件描述语言。

Chisel 的官方文档和其他资料可以在线获取:

- Chisel 主页是下载和学习 Chisel 的官方起点。
- Chisel Tutorial 提供了一个可直接使用的项目，包含带测试器和参考答案的小练习。

[3] 此功能需要 sbt 的 Eclipse 插件。

- Chisel Wiki 包含一份简短的 Chisel 用户指南以及进一步资料的链接。
- Chisel Testers 位于其仓库中，仓库内包含 Wiki 文档。
- Generator Bootcamp 是一门以硬件生成器为重点的 Chisel 课程，以 Jupyter notebook 形式提供。
- Christopher Celio 编写的 Chisel 风格指南。

2.7 练习

每一章都以一个动手练习结束。作为引言的练习，我们将使用一块 FPGA 开发板让一个 LED 闪烁。第一步，从 GitHub 克隆（或 fork）chisel-examples 仓库。Hello World 示例位于 `hello-world` 文件夹中，已配置为一个最小项目。你可以在 `src/main/scala/Hello.scala` 中查看闪烁 LED 的 Chisel 代码。按照以下步骤编译闪烁 LED：

```
$ git clone https://github.com/schoeberl/chisel-examples.git
$ cd chisel-examples/hello-world/
$ make
```

在初次下载 Chisel 组件之后，将生成 Verilog 文件 `Hello.v`。查看这个 Verilog 文件，你会看到它包含两个输入 `clock` 和 `reset`，以及一个输出 `io_led`。将这个 Verilog 文件与 Chisel 模块比较，你会发现 Chisel 模块中并没有 `clock` 或 `reset`。这些信号是隐式生成的；在大多数设计中，不必处理这些底层细节会更方便。Chisel 提供寄存器组件，它们会自动连接到 `clock` 和 `reset`（如果需要）。

下一步是为综合工具建立 FPGA 工程文件、分配引脚并编译[4] Verilog 代码，然后用生成的比特流文件配置 FPGA。我们无法提供这些步骤的详细说明，请查阅 Intel Quartus 或 Xilinx Vivado 工具的手册。不过，示例仓库在 `quartus` 文件夹中为几款常见 FPGA 开发板（例如 DE2-115）提供了可直接使用的 Quartus 工程。如果仓库支持你的开发板，请启动 Quartus，打开工程，点击 *Play* 按钮进行编译，再用 *Programmer* 按钮配置 FPGA 开发板，其中一个 LED 应该会开始闪烁。

恭喜！你已经成功让自己的第一个 Chisel 设计在 FPGA 上运行起来了！

如果 LED 没有闪烁，请检查复位信号的状态。在 DE2-115 的配置中，复位输入连接到了 SW0。

现在，把闪烁频率调慢或调快，然后重新构建。闪烁频率和闪烁模式可以传达不同的“情绪”。例如，缓慢闪烁的 LED 表示一切正常，快速闪烁的 LED 则表示报警状态。请尝试不同的频率，看看哪种频率最能表达这两种不同的情绪。

[4] 实际流程要复杂得多，包括综合逻辑、布局布线、时序分析以及生成比特流文件。不过，就这个入门示例而言，我们简单地称之为“编译”代码。

作为更具挑战性的扩展练习, 请生成以下闪烁模式: LED 每秒亮 200 ms。要实现这个模式, 你可能需要把 LED 状态的变化与计数器复位解耦，还需要第二个常量来决定何时改变 `blkReg` 寄存器的状态。这种模式会产生什么样的情绪？它更像警报，还是更像“心跳”信号?

第三章　基本组件

本章介绍数字设计的基本组件：组合电路和触发器。这些基本元件可以组合起来，构建更大、更有趣的电路。

3.1　信号类型与常量

Chisel 提供三种数据类型来描述信号、组合逻辑和寄存器：`Bits`、`UInt` 和 `SInt`。`UInt` 和 `SInt` 都继承自 `Bits`，这三种类型都表示一个位向量。`UInt` 赋予该位向量无符号整数的含义，`SInt` 则赋予有符号整数的含义。[1] Chisel 使用二进制补码作为有符号整数的表示方式。下面分别定义了 8 位的 `Bits`、8 位无符号整数和 10 位有符号整数：

```
Bits(8.W)
UInt(8.W)
SInt(10.W)
```

位向量的宽度由 Chisel 的宽度类型（`Width`）定义。下面的表达式把 Scala 整数 `n` 转换为 Chisel 的 `width`，用于定义 `Bits` 向量：

```
n.W
Bits(n.W)
```

常量可以通过把 Scala 整数转换为 Chisel 类型来定义：

```
0.U  // defines a UInt constant of 0
-3.S // defines a SInt constant of -3
```

常量也可以借助 Chisel 的宽度类型，在定义时指定位宽：

```
8.U(4.W) // An 4-bit constant of 8
```

如果你觉得 8.U 和 4.W 这种写法有点奇怪，可以把它看作带类型的整数常量。这种记法类似于 C、Java 和 Scala 中表示长整型常量的 8L。

Chisel 受益于 Scala 的类型推断，很多地方可以省略类型信息。位宽也是如此。在许多情况下，Chisel 会自动推断出正确的宽度。因此，用 Chisel 描述硬件比 VHDL 或 Verilog 更简洁、更易读。

对于十进制以外的常量，可以用字符串定义，并在前面加上 `h` 表示十六进制（基 16），`o` 表示八进制（基 8），`b` 表示二进制（基 2）。下面的例子用不同进制定义常量 255。这里省略了位宽，Chisel 会推断出能够容纳这些常量的最小宽度，本例中为 8 位。

```
"hff".U         // hexadecimal representation of 255
"o377".U        // octal representation of 255
"b1111_1111".U // binary representation of 255
```

[1] 当前版本的 Chisel 中，`Bits` 类型缺少一些运算，因此对用户代码来说用处不大。

上面的代码还展示了如何在表示常量的字符串中用下划线分组数字。下划线会被忽略。

为了表示逻辑值，Chisel 定义了 `Bool` 类型。`Bool` 可以表示 *true* 或 *false*。下面的代码定义了 `Bool` 类型以及 `Bool` 常量：把 Scala 的布尔常量 `true` 和 `false` 转换为 Chisel 的 `Bool` 常量。

```
Bool()
true.B
false.B
```

3.2 组合电路

Chisel 使用布尔代数运算符来描述组合电路，这些运算符与 C、Java、Scala 等编程语言中的定义相同：`&` 是与运算符，`|` 是或运算符。下面一行代码定义了一个电路：用与门组合信号 `a` 和 `b`，再用或门把结果与信号 `c` 组合。

```
val logic = (a & b) | c
```

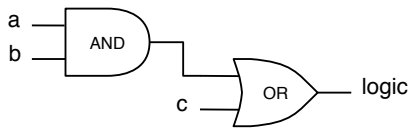


图 3.1：表达式 `(a & b) | c` 对应的逻辑。连线可以是单个位，也可以是多个位。Chisel 表达式与原理图表达的是同一个电路。

图 3.1 给出了这个组合表达式的原理图。注意，这个电路可能作用于位向量，而不只是用与门、或门组合的单根连线。

在这个例子中，我们没有定义信号 `logic` 的类型和宽度，二者都由表达式的类型和宽度推断得到。Chisel 中的标准逻辑运算如下：

```
val and = a & b // bitwise and
val or = a | b  // bitwise or
val xor = a ^ b // bitwise xor
val not = ~a    // bitwise negation
```

算术运算使用标准运算符：

```
val add = a + b // addition
val sub = a - b // subtraction
val neg = -a    // negate
val mul = a * b // multiplication
val div = a / b // division
val mod = a % b // modulo operation
```

运算结果的位置规则是：加法和减法取两个操作数位宽的最大值，乘法取两个位宽之和，除法和取模通常取被除数的位宽。[2]

信号也可以先定义为某种类型的 `Wire`，然后再用`:=` 更新运算符为它赋值。

```
val w = Wire(UInt())

w := a & b
```

单个位可以按如下方式提取：

```
val sign = x(31)
```

子字段可以从结束位置到起始位置提取：

```
val lowByte = largeWord(7, 0)
```

位域可以用 `Cat` 拼接。

```
val word = Cat(highByte, lowByte)
```

表 3.2 列出了完整的运算符（另见内置运算符）。Chisel 的运算符优先级由电路的求值顺序决定，遵循 Scala 的运算符优先级。如果有疑问，使用括号始终是良好的做法。[3]

表 3.2 列出了 Chisel 数据类型上定义的各种函数。

运算符	说明	数据类型
`* / %`	乘法、除法、取模	UInt, SInt
`+ -`	加法、减法	UInt, SInt
`=== =/=`	等于、不等于	UInt, SInt，返回 Bool
`> >= < <=`	比较	UInt, SInt，返回 Bool
`<< >>`	左移、右移（对 SInt 进行符号扩展）	UInt, SInt
`~`	按位取反	UInt, SInt, Bool
`& |^`	按位与、或、异或	UInt, SInt, Bool
`!`	逻辑非	Bool
`&& ||`	逻辑与、逻辑或	Bool

表 3.1： Chisel 定义的硬件运算符。

[2]精确的规则见 FIRRTL 规范。

[3]Chisel 的运算符优先级是硬件展开过程的副产品：执行 Scala 运算符时，会构建硬件节点树。Scala 的运算符优先级与 Java/C 类似，但并不完全相同。Verilog 的运算符优先级与 C 相同，VHDL 则不同。Verilog 为逻辑运算规定了优先级，而在 VHDL 中这些运算符优先级相同，从左到右求值。

函数	说明	数据类型
`v.andR v.orR v.xorR`	与、或、异或归约	UInt, SInt，返回 Bool
`v(n)`	提取单个位	UInt, SInt
`v(end, start)`	位域提取	UInt, SInt
`Fill(n, v)`	将位串重复 n 次	UInt, SInt
`Cat(a, b, ...)`	位域拼接	UInt, SInt

表 3.2: Chisel 定义的硬件函数，作用于 `v`。

3.2.1 多路选择器

多路选择器是一种在多个候选项中做出选择的电路。最基本的形式是在两个候选项中二选一。图 3.2 展示了一个这样的 2:1 多路选择器（简称 mux）。根据选择信号（`sel`）的取值，输出信号 `y` 将等于信号 `a` 或信号 `b`。

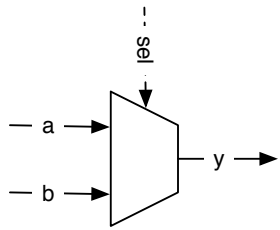


图 3.2: 一个基本的 2:1 多路选择器。

多路选择器可以用逻辑门搭建。不过，多路选择是如此常见的操作，因此 Chisel 直接提供了一个多路选择器：

```
val result = Mux(sel, a, b)
```

当 `sel` 为 `true.B` 时选择 `a`，否则选择 `b`。`sel` 的类型是 Chisel 的 `Bool`；输入 `a` 和 `b` 可以是任意 Chisel 基本类型或聚合类型（Bundle 或 Vec），只要二者类型相同即可。

有了逻辑运算、算术运算和多路选择器，就可以描述任何组合电路。不过，Chisel 还提供了更多组件和控制抽象，可以更优雅地描述组合电路，这些内容将在后续章节中介绍。

描述数字电路所需的第二个基本组件是状态元件，也称为寄存器，下面就来介绍。

3.3 寄存器

Chisel 提供了一个寄存器，它由一组 D 触发器构成。寄存器隐式连接到全局时钟，并在时钟上升沿更新。如果在声明寄存器时提供了初始值，它就会使用连接到全局复位信号的同步复位。寄存器可以是任何能够表示为位集合的 Chisel 类型。下面的代码定义了一个 8 位寄存器，复位时初始化为 0：

```
val reg = RegInit(0.U(8.W))
```

输入通过:= 更新运算符连接到寄存器，而寄存器的输出可以直接在表达式中按名称使用：

```
reg := d
val q = reg
```

寄存器也可以在定义时直接连接到它的输入：

```
val nextReg = RegNext(d)
```

寄存器还可以在定义时连接到输入，并以一个常量作为初始值：

```
val bothReg = RegNext(d, 0.U)
```

为了区分组合逻辑信号和寄存器，常见的做法是在寄存器名称后加后缀 `Reg`。另一种来自 Java 和 Scala 的常见做法是，对由多个单词组成的标识符使用驼峰命名法。约定是：函数和变量以小写字母开头，类（类型）以大写字母开头。

3.3.1 计数

计数是数字系统中的基本操作。人们有时用它来统计事件，但更常见的是用计数来定义时间间隔：对时钟周期计数，并在时间间隔到期时触发某个动作。

一种简单的做法是计数到某个值。不过在计算机科学和数字设计中，计数从 0 开始。因此，如果要数到 10，实际上是从 0 数到 9。下面的代码展示了一个计数器：数到 9，到达 9 后回绕到 0。

```
val cntReg = RegInit(0.U(8.W))

cntReg := Mux(cntReg === 9.U, 0.U, cntReg + 1.U)
```

3.4 使用 Bundle 和 Vec 组织信号

Chisel 提供两种结构来组织相关信号：（1）`Bundle`，用于组合不同类型的信号；（2）`Vec`，用于表示可按下标索引的同类型信号集合。`Bundle` 和 `Vec` 可以任意嵌套。

Chisel 的 Bundle 将若干信号组合在一起。既可以整体引用整个 Bundle，也可以通过字段名访问各个字段。定义一个 Bundle（信号集合）的方法是：定义一个继承 `Bundle` 的类，并在构造器块中把各字段列为 `val`。

```
class Channel() extends Bundle {
  val data = UInt(32.W)
  val valid = Bool()
}
```

使用 Bundle 时，我们用 `new` 创建它，并包装进 `Wire`。字段通过点号访问：

```
  val ch = Wire(new Channel())
  ch.data := 123.U
  ch.valid := true.B

  val b = ch.valid
```

点号表示法在面向对象语言中很常见：`x.y` 表示 `x` 是一个对象引用，`y` 是该对象的一个字段。由于 Chisel 是面向对象的，我们使用点号访问 Bundle 中的字段。Bundle 类似于 C 中的 `struct`、VHDL 中的 `record`，或 SystemVerilog 中的 `struct`。Bundle 也可以整体引用：

```
  val channel = ch
```

Chisel 的 `Vec` 表示一组同类型信号（一个向量）。每个元素都可以通过下标访问。Chisel 的 `Vec` 类似于其他编程语言中的数组数据结构。[4] 创建 `Vec` 时，构造器需要两个参数：元素个数和元素类型。组合类型的 `Vec` 需要包装进 `Wire`：

```
  val v = Wire(Vec(3, UInt(4.W)))
```

各个元素通过 `(index)` 访问。

```
  v(0) := 1.U
  v(1) := 3.U
  v(2) := 5.U

  val idx = 1.U(2.W)
  val a = v(idx)
```

包装进 `Wire` 的向量就是一个多路选择器。我们也可以把向量包装进寄存器，从而定义寄存器数组。下面的例子为处理器定义了一个寄存器堆：32 个 32 位寄存器，类似于经典的 32 位 RISC 处理器，例如 32 位版本的 RISC-V。

```
  val registerFile = Reg(Vec(32, UInt(32.W)))
```

寄存器堆中的元素通过下标访问，并像普通寄存器一样使用。

[4] `Array` 这个名字已经被 Scala 使用了。

```
registerFile(idx) := dIn
val dOut = registerFile(idx)
```

我们可以自由地混合使用 Bundle 和 Vec。创建 Bundle 类型的向量时，需要为向量字段传入一个原型。使用前面定义的 `Channel`，可以这样创建一个通道向量：

```
val vecBundle = Wire(Vec(8, new Channel()))
```

Bundle 中也可以包含 Vec：

```
class BundleVec extends Bundle {
  val field = UInt(8.W)
  val vector = Vec(4,UInt(8.W))
}
```

如果需要 Bundle 类型的寄存器并提供复位值，我们首先创建该 Bundle 的 `Wire`，按需设置各个字段，然后把这个 Bundle 传给 `RegInit`：

```
val initVal = Wire(new Channel())

initVal.data := 0.U
initVal.valid := false.B

val channelReg = RegInit(initVal)
```

通过 `Bundle` 和 `Vec` 的组合，我们可以定义自己的数据结构，它们是非常强大的抽象。

3.5 Chisel 生成硬件

看过一些初步的 Chisel 代码后，它可能看起来与 Java 或 C 等经典编程语言相似。然而，Chisel（以及任何硬件描述语言）定义的是硬件组件。在软件程序中，代码一行接一行地执行；而在硬件中，所有代码行并行执行。

必须牢记：Chisel 代码会生成硬件。请试着想象，或者在纸上画出你的 Chisel 电路描述所生成的各个模块。每次创建组件都会增加硬件；每条赋值语句都会生成门电路和/或触发器。

更技术性地说，Chisel 执行你的代码时，它作为一个 Scala 程序运行；通过执行 Chisel 语句，它收集硬件组件并连接这些节点。这个硬件节点网络就是硬件，它可以输出用于 ASIC 或 FPGA 综合的 Verilog 代码，也可以用 Chisel 测试器进行测试。硬件节点网络中的一切完全并行执行。

对软件工程师来说，可以想象这种巨大的并行性：在硬件中创建并行执行，不需要把应用划分成线程，也不需要为通信正确地加锁。

3.6 练习

在引言中，你在 FPGA 开发板上实现了一个闪烁 LED （来自 chisel-examples），这是一个合理的硬件 *Hello World* 示例。它只使用了内部状态、一个 LED 输出，没有输入。把这个项目复制到一个新文件夹，并在 `io Bundle` 中增加一些输入，例如 `val sw = Input(UInt(2.W))`：

```
val io = IO(new Bundle {
  val sw = Input(UInt(2.W))
  val led = Output(UInt(1.W))
})
```

对于这些开关，还需要为 FPGA 开发板分配引脚名称。你可以在 ALU 工程的 Quartus 工程文件中找到引脚分配示例（例如，DE2-115 FPGA 开发板的示例）。

定义好这些输入和引脚分配后，先做一个简单的测试：从设计中删除所有闪烁逻辑，把一个开关连接到 LED 输出；编译并配置 FPGA。你能用开关控制 LED 的亮灭吗？如果能，说明输入已经可以正常工作；如果不能，就需要调试 FPGA 配置。引脚分配也可以通过工具的图形界面完成。

现在使用两个开关，实现一种基本的组合逻辑功能，例如对两个开关做与运算，并把结果显示在 LED 上。然后换一种功能。下一步使用三个输入开关实现一个多路选择器：一个开关作为选择信号，另外两个作为 2:1 多路选择器的两个输入。

现在你已经能够在 FPGA 中实现简单的组合逻辑功能并在真实硬件上测试它们。下一步，我们将初步了解构建过程如何生成 FPGA 配置，并探索 Chisel 的简单测试框架，它使你无需配置 FPGA、无需拨动开关就能测试电路。

第四章　构建流程与测试

要开始编写更有趣的 Chisel 代码，首先需要学习如何编译 Chisel 程序、如何生成可在 FPGA 中执行的 Verilog 代码，以及如何编写测试来调试电路并验证其正确性。

Chisel 是用 Scala 编写的，因此任何支持 Scala 的构建流程都可以用于 Chisel 项目。Scala 的一种常用构建工具是 sbt，它是 Scala 交互式构建工具（Scala interactive build tool）的缩写。除了驱动构建和测试流程，`sbt` 还会下载正确版本的 Scala 和 Chisel 库。

4.1　使用 sbt 构建项目

表示 Chisel 的 Scala 库和 Chisel 测试器会在构建过程中自动从 Maven 仓库下载。这些库通过 `build.sbt` 引用。可以把 `build.sbt` 中的版本配置为 `latest.release`，以始终使用最新的 Chisel 版本。但这意味着每次构建时都要从 Maven 仓库查询版本，而该查询需要联网，构建才能成功。更好的做法是在 `build.sbt` 中为 Chisel 和所有其他 Scala 库指定明确的版本。有时，能够离线编写和测试硬件代码也很不错，例如在飞机上做硬件设计。

4.1.1　源码组织

`sbt` 沿用 Maven 构建自动化工具的源码约定。Maven 还负责组织开源 Java 库的仓库。[1]

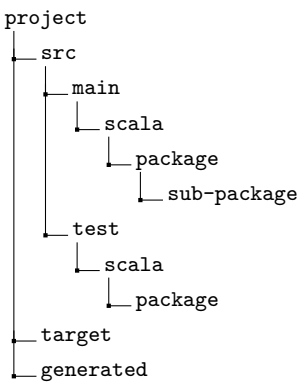


图 4.1：Chisel 项目（使用 `sbt`）的源码树

图 4.1 展示了一个典型 Chisel 项目的源码树结构。项目的根目录是项目主目录，其中包含 `build.sbt`，还可能包含用于构建流程的 `Makefile`、README 和 LICENSE 文件。`src` 文件夹

[1] 你第一次构建时下载 Chisel 库的地方也正是这里：`https://mvnrepository.com/artifact/edu.berkeley.cs/chisel3`。

包含所有源代码，并分为两部分：`main` 存放硬件源码，`test` 存放测试器。Chisel 继承自 Scala，Scala 又继承自 Java，因此源码按包组织。包把 Chisel 代码组织到命名空间中。包中还可以包含子包。`target` 文件夹包含类文件和其他生成文件。我还建议为生成的 Verilog 文件使用一个文件夹，通常命名为 `generated`。

要使用 Chisel 的命名空间功能，需要声明某个类/模块定义在某个包中，本例中为 `mypacket`：

```
package mypack

import chisel3._

class Abc extends Module {
  val io = IO(new Bundle{})
}
```

注意，这个例子导入了 `chisel3` 包，以便使用 Chisel 的类。

要在其他上下文（命名空间）中使用模块 `Abc`，需要导入 `mypacket` 包中的组件。下划线（_）是通配符，表示导入 `mypacket` 中的所有类。

```
import mypack._

class AbcUser extends Module {
  val io = IO(new Bundle{})

  val abc = new Abc()

}
```

也可以不导入 `mypacket` 中的所有类型，而是用完全限定名 `mypack.Abc` 来引用 `mypack` 包中的模块 `Abc`。

```
class AbcUser2 extends Module {
  val io = IO(new Bundle{})

  val abc = new mypack.Abc()

}
```

还可以只导入单个类并创建它的实例：

```
import mypack.Abc

class AbcUser3 extends Module {
  val io = IO(new Bundle{})

  val abc = new Abc()

}
```

4.1.2 运行 sbt

可以用一条简单的 `sbt` 命令编译并执行 Chisel 项目：

```
$ sbt run
```

该命令会编译源码树中的所有 Chisel 代码，并查找包含 `main` 方法的 `object`，或者更简单的、继承 `App` 的 `object`。如果有多个这样的对象，`sbt` 会列出所有对象供你选择。也可以直接把要执行的对象作为参数传给 `sbt`：

```
$ sbt "runMain mypacket.MyObject"
```

默认情况下，`sbt` 只搜索源码树的 `main` 部分，而不搜索 `test` 部分。[2] 不过，本文介绍的 Chisel 测试器也包含 `main`，但应放在源码树的 `test` 部分。要执行测试器树中的 `main`，请使用以下 `sbt` 命令：

```
$ sbt "test:runMain mypacket.MyTester"
```

现在我们已经了解了 Chisel 项目的基本结构，以及如何用 `sbt` 编译和运行它，接下来学习一个简单的测试框架。

4.2 使用 Chisel 进行测试

硬件设计的测试通常称为测试平台。测试平台实例化被测设计（DUT），驱动输入端口，观察输出端口，并与期望值进行比较。

Chisel 以 `PeekPokeTester` 的形式提供测试平台。Chisel 的一个优势是可以利用 Scala 的全部能力来编写测试平台。例如，可以在软件模拟器中实现硬件的预期功能，再把硬件仿真结果与软件模拟结果进行比较。在测试处理器实现时，这种方法非常高效 [6]。

要使用 `PeekPokeTester`，需要导入以下包：

```
import chisel3._
import chisel3.iotesters._
```

测试电路至少包含三个部分：（1）被测设备（通常称为 DUT）；（2）测试逻辑，也称为测试平台；（3）包含 `main` 函数、用于启动测试的测试器对象。

下面的代码展示了我们简单的被测设计。它有两个输入端口和一个输出端口，位宽均为 2 位。该电路对输入做按位与运算，并在输出端返回结果：

[2] 这是 Java/Scala 的约定：test 文件夹存放单元测试，而不是带 `main` 的对象。

```
class DeviceUnderTest extends Module {
  val io = IO(new Bundle {
    val a = Input(UInt(2.W))
    val b = Input(UInt(2.W))
    val out = Output(UInt(2.W))
  })

  io.out := io.a & io.b
}
```

这个 DUT 的测试平台继承 `PeekPokeTester`，并把 DUT 作为构造器参数：

```
class TesterSimple(dut: DeviceUnderTest) extends PeekPokeTester(dut) {

  poke(dut.io.a, 0.U)
  poke(dut.io.b, 1.U)
  step(1)
  println("Result is: " + peek(dut.io.out).toString)
  poke(dut.io.a, 3.U)
  poke(dut.io.b, 2.U)
  step(1)
  println("Result is: " + peek(dut.io.out).toString)
}
```

`PeekPokeTester` 可以用 `poke()` 设置输入值，用 `peek()` 读回输出值。测试器通过 `step(1)` 让仿真前进一步（即一个时钟周期）。可以用 `println()` 打印输出值。

测试由下面的测试器 main 创建并运行：

```
object TesterSimple extends App {
  chisel3.iotesters.Driver(() => new DeviceUnderTest()) { c =>
    new TesterSimple(c)
  }
}
```

运行测试时，可以在终端看到打印的结果（以及其他信息）：

```
[info] [0.004] SEED 1544207645120
[info] [0.008] Result is: 0
[info] [0.009] Result is: 2
test DeviceUnderTest Success: 0 tests passed in 7 cycles
taking 0.021820 seconds
[info] [0.010] RAN 2 CYCLES PASSED
```

可以看到，0 与 1 的结果是 0；3 与 2 的结果是 2。除了人工检查打印结果（这是一个很好的起点），我们还可以在测试平台中用 `expect()` 表达预期，把输出端口和期望值作为参数。下面的例子展示了如何用 `expect()` 进行测试：

```
class Tester(dut: DeviceUnderTest) extends PeekPokeTester(dut) {
```

```
  poke(dut.io.a, 3.U)
  poke(dut.io.b, 1.U)
  step(1)
  expect(dut.io.out, 1)
  poke(dut.io.a, 2.U)
  poke(dut.io.b, 0.U)
  step(1)
  expect(dut.io.out, 0)
}
```

执行这个测试不会打印硬件输出的值，但会显示所有测试都通过了，因为所有期望值都正确。

```
[info] [0.001] SEED 1544208437832
test DeviceUnderTest Success: 2 tests passed in 7 cycles
taking 0.018000 seconds
[info] [0.009] RAN 2 CYCLES PASSED
```

如果 DUT 或测试平台中存在错误，测试就会失败，并输出一条错误消息，说明期望值与实际值之间的差异。下面我们把测试平台改为期望 4，这会产生一个错误：

```
[info] [0.002] SEED 1544208642263
[info] [0.011] EXPECT AT 2   io_out got 0 expected 4 FAIL
test DeviceUnderTest Success: 1 tests passed in 7 cycles
taking 0.022101 seconds
[info] [0.012] RAN 2 CYCLES FAILED FIRST AT CYCLE 2
```

本节介绍了 Chisel 用于简单测试的基本测试设施。不过，在 Chisel 中还可以利用 Scala 的全部能力来编写测试器。

4.2.1 使用 ScalaTest

ScalaTest 是一个用于 Scala（和 Java）的测试工具，我们可以用它来运行 Chisel 测试器。要使用它，请在 `build.sbt` 中加入以下依赖：

```
libraryDependencies += "org.scalatest" %% "scalatest" % "3.0.5" % "test"
```

测试通常放在 `src/test/scala` 中，可以用以下命令运行：

```
$ sbt test
```

下面是一个最小的测试（测试界的 hello world），用于测试 Scala 整数加法：

```
import org.scalatest._

class ExampleSpec extends FlatSpec with Matchers {

  "Integers" should "add" in {
    val i = 2
    val j = 3
    i + j should be (5)
  }
}
```

虽然 Chisel 测试比 Scala 程序的单元测试更重量级，但我们仍可以把 Chisel 测试包装进 ScalaTest 类中。对于前面展示的 `Tester`，可以这样包装：

```
class SimpleSpec extends FlatSpec with Matchers {

  "Tester" should "pass" in {
    chisel3.iotesters.Driver(() => new DeviceUnderTest()) { c =>
      new Tester(c)
    } should be (true)
  }
}
```

这个练习的主要好处是可以用一条简单的 `sbt test` 运行所有测试（而不必运行 `main`）。也可以用 `sbt` 只运行单个测试：

```
$ sbt "testOnly SimpleSpec"
```

4.3 练习

在本练习中，我们将重新审视 chisel-examples 中的闪烁 LED，并探索 Chisel 测试。

4.3.1 一个最小项目

首先，我们来弄清楚什么是最小的 Chisel 项目。浏览 Hello World 示例中的文件。`Hello.scala` 是唯一的硬件源文件，其中包含闪烁 LED 的硬件描述（`class Hello`），以及一个生成 Verilog 代码的 `App`。

每个文件都以导入 Chisel 和相关包开始：

```
import chisel3._
```

接下来是硬件描述，如代码清单 2.1 所示。要生成 Verilog 描述，需要一个应用程序。继承 `App` 的 Scala `object` 就是一个应用程序，它会隐式生成程序入口所在的 main 函数。这个应用程序唯一的动作是创建一个新的 `HelloWorld` 对象，并把它传给 Chisel 驱动函数 `execute`。第一个

参数是字符串数组，可以在其中设置构建选项（例如输出文件夹）。下面的代码将生成 Verilog 文件 `Hello.v`。

```
object Hello extends App {
  chisel3.Driver.execute(Array[String](), () => new Hello())
}
```

用以下命令手动运行该示例的生成过程：

```
$ sbt "runMain Hello"
```

然后用编辑器查看生成的 `Hello.v`。生成的 Verilog 代码可能不太易读，但我们仍能看出一些细节。文件以一个名为 `Hello` 的模块开始，与我们的 Chisel 模块同名。可以找到我们的 LED 端口，写作 `output io_led`。引脚名就是 Chisel 中的名称加上前缀 `io_`。除了 LED 引脚，模块还包含 `clock` 和 `reset` 两个输入信号，这两个信号由 Chisel 自动添加。

此外，还可以找到两个寄存器 `cntReg` 和 `blkReg` 的定义，以及模块定义末尾这两个寄存器的复位和更新逻辑。注意，Chisel 生成的是同步复位。

为了让 `sbt` 能够获取正确的 Scala 编译器和 Chisel 库，需要一个 `build.sbt`：

```
scalaVersion := "2.11.7"

resolvers ++= Seq(
  Resolver.sonatypeRepo("snapshots"),
  Resolver.sonatypeRepo("releases")
)

libraryDependencies += "edu.berkeley.cs" %% "chisel3" % "3.1.2"
```

注意，这个例子使用了明确的 Chisel 版本号，以免每次运行都检查新版本（如果没有联网，例如在飞行途中做硬件设计，这种检查会失败）。把库依赖改为以下内容，即可让 `build.sbt` 使用最新的 Chisel 版本：

```
libraryDependencies += "edu.berkeley.cs" %% "chisel3" % "latest.release"
```

然后用 `sbt` 重新构建。是否有更新的 Chisel 版本？它会自动下载吗？

为了方便，该项目还包含一个 `Makefile`。它只包含 `sbt` 命令，这样我们不必记住命令，只需运行以下命令即可生成 Verilog 代码：

```
make
```

除了 `README` 文件，示例项目还包含面向不同 FPGA 开发板的工程文件。例如，在 quartus/altde2-115 中可以找到为 DE2-115 开发板定义 Quartus 工程的两个文件。主要定义（源文件、器件、引脚分配）可以在纯文本文件 hello.qsf 中找到。浏览该文件，找出哪些引脚连接到哪些信号。如果需要把工程移植到其他开发板，修改就在这里进行。如果安装了 Quartus，

请打开该工程，点击绿色的 *Play* 按钮编译，然后配置 FPGA。

注意，*Hello World* 是一个最小的 Chisel 项目。更真实的项目会把源文件组织到包中，并包含测试器。下一个练习将探索这样的项目。

4.3.2 测试练习

在上一章的练习中，你扩展了闪烁 LED 示例，增加了一些输入，实现了与门和多路选择器，并在 FPGA 上运行。现在我们将使用这个例子，用 Chisel 测试器测试其功能，从而实现自动化测试，并摆脱对 FPGA 的依赖。使用上一章的设计，添加一个 Chisel 测试器来测试其功能。尝试枚举所有可能的输入，并用 `expect()` 测试输出。

在 Chisel 中进行测试可以加快设计的调试速度。不过，将设计综合到 FPGA 上并进行测试始终是个好主意。这样可以实际检查设计的规模（通常以 LUT 和触发器计）以及设计在最高时钟频率下的性能。作为参考，一个教科书式的流水线 RISC 处理器可能占用约 3000 个 4 输入 LUT，在低成本 FPGA（Intel Cyclone 或 Xilinx Spartan）上可能运行在 100 MHz 左右。

第五章　组件

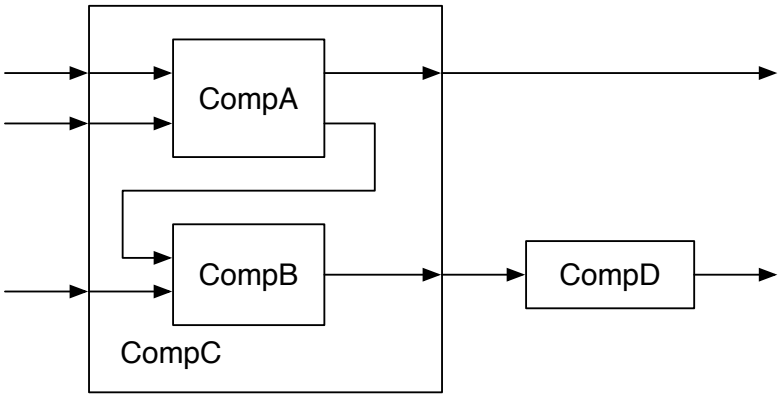


图 5.1：由层次化组件构成的设计。

较大的数字设计通常按层次结构划分为一组组件。每个组件都有一个由输入、输出连线组成的接口，通常称为端口。这些端口类似于集成电路（IC）上的输入、输出引脚。组件通过连接输入和输出进行组装。组件可以包含子组件，从而构成层次结构。最外层的组件与芯片上的物理引脚相连，称为顶层组件。

图 5.1 展示了一个示例设计。组件 C 有三个输入端口和两个输出端口。组件 C 本身由两个子组件 A 和 B 组装而成，它们分别连接到 C 的输入和输出。A 的一个输出连接到 B 的一个输入。组件 D 与组件 C 处于同一层次，并与 C 相连。

本章将说明如何在 Chisel 中描述组件，并给出若干标准组件的示例。这些标准组件有两个作用：（1）作为 Chisel 代码示例；（2）构成一个组件库，可直接在你的设计中复用。

5.1　Chisel 中的组件就是模块

硬件组件在 Chisel 中称为模块（Module）。每个模块都继承 `Module` 类，并包含一个用于接口的字段 `io`。接口由一个 `Bundle` 定义，并包装在 `IO()` 调用中。`Bundle` 中的字段表示模块的输入和输出端口。字段的方向通过把它包装进 `Input()` 或 `Output()` 调用来指定。方向是从组件自身的视角来看的。

下面的代码定义了图 5.1 中的两个示例组件 A 和 B：

```
class CompA extends Module {
  val io = IO(new Bundle {
    val a = Input(UInt(8.W))
    val b = Input(UInt(8.W))
```

```
    val x = Output(UInt(8.W))
    val y = Output(UInt(8.W))
  })

  // function of A
}

class CompB extends Module {
  val io = IO(new Bundle {
    val in1 = Input(UInt(8.W))
    val in2 = Input(UInt(8.W))
    val out = Output(UInt(8.W))
  })

  // function of B
}
```

组件 A 有两个输入 a 和 b，以及两个输出 x 和 y。组件 B 的端口命名为 in1、in2 和 out。所有端口都使用 8 位无符号整数（UInt）。由于这段示例代码的重点是连接组件、构建层次结构，因此没有展示组件内部的实现。组件的实现写在注释“X 的功能”所在的位置。由于这些示例组件没有具体功能，我们使用了通用的端口名。在实际设计中，应使用有描述性的端口名，例如 data、valid 或 ready。

组件 C 有三个输入端口和两个输出端口，由组件 A 和 B 构成。下面展示 A、B 如何连接到 C 的端口，以及 A 的一个输出端口如何连接到 B 的一个输入端口：

```
class CompC extends Module {
  val io = IO(new Bundle {
    val in_a = Input(UInt(8.W))
    val in_b = Input(UInt(8.W))
    val in_c = Input(UInt(8.W))
    val out_x = Output(UInt(8.W))
    val out_y = Output(UInt(8.W))
  })

  // create components A and B
  val compA = Module(new CompA())
  val compB = Module(new CompB())

  // connect A
  compA.io.a := io.in_a
  compA.io.b := io.in_b
  io.out_x := compA.io.x
  // connect B
  compB.io.in1 := compA.io.y
  compB.io.in2 := io.in_c
  io.out_y := compB.io.out
}
```

组件用 new 创建，例如 new CompA()，并需要包装进 Module() 调用。对该模块的引用保存在局部变量中，本例中为 val compA = Module(new CompA())。

通过这个引用，可以访问模块的 `io` 字段以及 IO `Bundle` 的各个字段，从而访问各个 IO 端口。

我们设计中最简单的组件只有一个输入端口 `in` 和一个输出端口 `out`：

```
class CompD extends Module {
  val io = IO(new Bundle {
    val in = Input(UInt(8.W))
    val out = Output(UInt(8.W))
  })

  // function of D
}
```

示例设计最后缺少的部分是顶层组件，它由组件 C 和 D 组装而成：

```
class TopLevel extends Module {
  val io = IO(new Bundle {
    val in_a = Input(UInt(8.W))
    val in_b = Input(UInt(8.W))
    val in_c = Input(UInt(8.W))
    val out_m = Output(UInt(8.W))
    val out_n = Output(UInt(8.W))
  })

  // create C and D
  val c = Module(new CompC())
  val d = Module(new CompD())

  // connect C
  c.io.in_a := io.in_a
  c.io.in_b := io.in_b
  c.io.in_c := io.in_c
  io.out_m := c.io.out_x
  // connect D
  d.io.in := c.io.out_y
  io.out_n := d.io.out
}
```

良好的组件设计类似于软件设计中良好的函数或方法设计。主要问题之一是：一个组件应该包含多少功能，组件应该有多大。两个极端是极小的组件（例如加法器）和巨大的组件（例如完整的微处理器）。

硬件设计的初学者常常从极小的组件开始。问题在于，数字设计教材为了说明原理，往往使用很小的组件；这些教材（包括本书）中的示例之所以小，是为了能放在一页内，并且不被过多细节干扰。

组件的接口略显繁琐（需要写类型、名称、方向和 IO 构造）。根据经验，我建议组件的核心功能部分至少应与组件接口一样长。

对于计数器之类的极小组件，Chisel 提供了一种更轻量的描述方式：把它们写成返回硬件的函数。

5.2 算术逻辑单元

对于执行运算的电路（例如微处理器），核心组件之一是算术逻辑单元，简称 ALU。图 5.2 展示了 ALU 的符号。

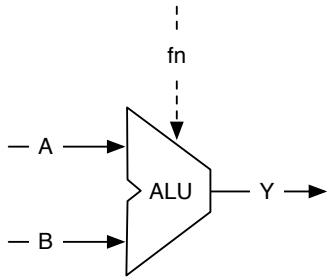


图 5.2: 算术逻辑单元，简称 ALU。

ALU 有两个数据输入（图中标为 `A` 和 `B`）、一个功能输入 `fn`，以及一个输出（标为 Y）。ALU 对 `A` 和 `B` 进行运算，并在输出端给出结果。输入 `fn` 选择对 `A` 和 `B` 执行的操作。这些操作通常包括加法、减法等算术运算，以及与、或、异或等逻辑功能。这就是它被称为算术逻辑单元的原因。

功能输入 `fn` 选择操作。ALU 通常是不含任何状态元件的组合电路。ALU 也可能有额外的输出，用于指示结果的某些性质，例如是否为零或符号。

下面的代码展示了一个 16 位输入、输出的 ALU，支持加、减、或、与运算，由 2 位的 `fn` 信号选择。

```
class Alu extends Module {
  val io = IO(new Bundle {
    val a = Input(UInt(16.W))
    val b = Input(UInt(16.W))
    val fn = Input(UInt(2.W))
    val y = Output(UInt(16.W))
  })

  // some default value is needed
  io.y := 0.U

  // The ALU selection
  switch(io.fn) {
    is(0.U) { io.y := io.a + io.b }
    is(1.U) { io.y := io.a - io.b }
    is(2.U) { io.y := io.a | io.b }
    is(3.U) { io.y := io.a & io.b }
  }
}
```

这个例子使用了 Chisel 的一个新结构：`switch/is`，用来描述选择 ALU 输出的表。要使用这个工具函数，需要导入另一个 Chisel 包：

```
import chisel3.util._
```

5.3 批量连接

对于具有多个 IO 端口的组件，Chisel 提供了批量连接运算符 <>。该运算符按名称双向连接 Bundle 中对应的字段。Chisel 使用叶子字段的名称进行连接；如果名称不匹配，则不会连接。

举个例子，假设我们要构建一个流水线处理器。取指阶段有如下接口：

```
class Fetch extends Module {
  val io = IO(new Bundle {
    val instr = Output(UInt(32.W))
    val pc = Output(UInt(32.W))
  })
  // ... Implementation od fetch
}
```

下一个阶段是译码阶段。

```
class Decode extends Module {
  val io = IO(new Bundle {
    val instr = Input(UInt(32.W))
    val pc = Input(UInt(32.W))
    val aluOp = Output(UInt(5.W))
    val regA = Output(UInt(32.W))
    val regB = Output(UInt(32.W))
  })
  // ... Implementation of decode
}
```

我们这个简单处理器的最后一个阶段是执行阶段。

```
class Execute extends Module {
  val io = IO(new Bundle {
    val aluOp = Input(UInt(5.W))
    val regA = Input(UInt(32.W))
    val regB = Input(UInt(32.W))
    val result = Output(UInt(32.W))
  })
  // ... Implementation of execute
}
```

要连接这三个阶段，只需两个 <> 运算符。我们也可以把子模块的端口与父模块连接起来。

```
  val fetch = Module(new Fetch())
  val decode = Module(new Decode())
```

```
val execute = Module(new Execute)

fetch.io <> decode.io
decode.io <> execute.io
io <> execute.io
```

5.4 用函数实现轻量级组件

模块是组织硬件描述的通用方式。但声明模块、实例化模块并连接它们时，需要写一些样板代码。组织硬件的轻量方式是使用函数。Scala 函数可以接收 Chisel（和 Scala）参数，并返回生成的硬件。作为一个简单的例子，我们生成一个加法器：

```
def adder (x: UInt, y: UInt) = {
  x + y
}
```

然后只需调用函数 `adder`，就可以创建两个加法器。

```
val x = adder(a, b)
// another adder
val y = adder(c, d)
```

注意，这是一个硬件生成器。在展开（elaboration）过程中，你并没有执行任何加法操作，而是创建了两个加法器（两个硬件实例）。加法器只是一个为了保持简单的示例。Chisel 已经提供了加法器生成函数，例如 `+(that: UInt)`。

作为轻量级硬件生成器的函数也可以包含状态（包括寄存器）。下面的例子返回一个单时钟周期延迟元件（一个寄存器）。如果函数只有一条语句，可以写成一行，并省略花括号 ()。

```
def delay(x: UInt) = RegNext(x)
```

把函数本身作为参数再次调用，就生成了一个两时钟周期延迟。

```
val delOut = delay(delay(delIn))
```

同样，这个例子太短，没有太大实用价值，因为 `RegNext()` 本身就能创建用于延迟的寄存器。

函数可以声明为 `Module` 的一部分。不过，要在不同模块中使用的函数，最好放在一个收集工具函数的 Scala object 中。

第六章　组合逻辑构建模块

本章探讨各种组合电路，它们是构建更复杂系统的基本模块。原则上，所有组合电路都可以用布尔方程描述。不过，更常见、更高效的方式是用表格来描述，然后让综合工具提取并最小化布尔方程。译码器和编码器就是两个最适合用表格描述的基本电路。

6.1　组合电路

在介绍标准组合逻辑模块之前，我们先看看如何在 Chisel 中表达组合电路。最简单的形式是布尔表达式，可以为它指定一个名称：

```
val e = (a & b) | c
```

把布尔表达式赋给一个 Scala 值，就为它指定了名称（e）。该表达式可以在其他表达式中复用：

```
val f = ~e
```

这样的表达式被视为固定值。用 = 重新赋值 e 会导致 Scala 编译错误：reassignment to val。如果改用 Chisel 的:= 运算符，如下所示：

```
e := c & b
```

则会在运行时抛出异常：Cannot reassign to read-only。

Chisel 也支持用条件更新来描述组合电路。这类电路先声明为 Wire，然后用 when 等条件操作来描述电路逻辑。下面的代码声明了一个 UInt 类型的 Wire w，并赋予默认值 0。when 块接收一个 Chisel Bool；当 cond 为 true.B 时，把 3 赋给 w。

```
val w = Wire(UInt())

w := 0.U
when (cond) {
  w := 3.U
}
```

这个电路的逻辑是一个多路选择器：两个输入分别是常量 0 和 3，条件 cond 是选择信号。请记住，我们描述的是硬件电路，而不是带条件执行的软件程序。

Chisel 的条件结构 when 也有类似 *else* 的形式，称为 otherwise。如果在所有条件下都赋值，就可以省略默认值赋值：

```
val w = Wire(UInt())

when (cond) {
  w := 1.U
} otherwise {
  w := 2.U
}
```

Chisel 还支持用.elsewhen 构成条件链（if/elseif/else 链）：

```
val w = Wire(UInt())

when (cond) {
  w := 1.U
} .elsewhen (cond2) {
  w := 2.U
} otherwise {
  w := 3.U
}
```

注意.elsewhen 前面的点号“.”，这是在 Scala 中串联方法所必需的。.elsewhen 分支可以任意长。不过，如果条件链只取决于单个信号，最好使用 switch 语句。下一小节将结合译码器电路介绍该语句。

对于更复杂的组合电路，为 Wire 赋予默认值可能很实用。默认赋值可以与 Wire 声明合并，使用 WireDefault。[1]

```
val w = WireInit(0.U)

when (cond) {
  w := 3.U
}
// ... and some more complex conditional assignments
```

有人可能会问：Scala 已经有 if、else if 和 else，为什么还要用 when、.elsewhen 和 otherwise？Scala 的语句用于有条件地执行 Scala 代码，而不是生成 Chisel（多路选择器）硬件。Scala 条件语句在 Chisel 中也有用处：编写电路生成器时，可以用参数有条件地生成不同的硬件实例。

6.2 译码器

译码器把 n 位二进制数转换为 m 位信号，其中 $m \leq 2^n$。输出采用独热编码（恰好有一位为 1）。

图 6.1 展示了一个 2 位到 4 位的译码器。译码器的功能可以用真值表描述，如表 6.2 所示。

Chisel 的 switch 语句可以用真值表的形式描述逻辑。switch 语句不属于 Chisel 核心语言的一部分，因此需要引入 chisel.util 包中的内容。

```
import chisel3.util._
```

下面的代码使用 Chisel 的 switch 语句描述一个译码器：

```
result := 0.U
```

[1] 在当前版本的 Chisel 中它叫 WireInit，但在 Chisel 3.2 发布后将改为 WireDefault。

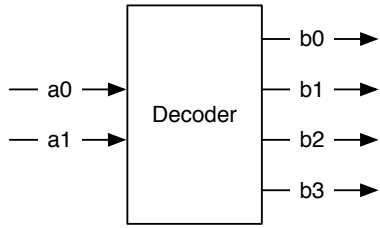


图 6.1： 2 位到 4 位译码器。

a	b
00	0001
01	0010
10	0100
11	1000

表 6.1： 2 到 4 译码器的真值表。

```
switch(sel) {
  is (0.U) { result := 1.U}
  is (1.U) { result := 2.U}
  is (2.U) { result := 4.U}
  is (3.U) { result := 8.U}
}
```

上面的 `switch` 语句列出了 `sel` 信号所有可能的取值，并把译码结果赋给 `result` 信号。注意，即使我们枚举了所有可能的输入值，Chisel 仍要求赋一个默认值，这里把 0 赋给 `result`。这个默认赋值永远不会生效，因此会被后端工具优化掉。它的作用是避免组合电路（Chisel 中的 `Wire`）出现不完整赋值，否则在 VHDL 和 Verilog 等硬件描述语言中会生成意料之外的锁存器。Chisel 不允许不完整赋值。

在前面的例子中，我们对信号使用了无符号整数。对于编码电路，用二进制记法可能更清晰：

```
switch (sel) {
  is ("b00".U) { result := "b0001".U}
  is ("b01".U) { result := "b0010".U}
  is ("b10".U) { result := "b0100".U}
  is ("b11".U) { result := "b1000".U}
}
```

用表格表示译码器功能非常易读，但也略显冗长。观察表格可以发现一种规律：把 1 左移 `sel` 所表示的位数。因此，可以用 Chisel 的左移操作 `<<` 来表达译码器。

```
result := 1.U << sel
```

译码器可以用作多路选择器的构建模块：把译码器输出作为使能信号，与多路选择器的数据输入进行与运算。不过，在 Chisel 中不必自己搭建多路选择器，因为核心库已经提供了 `Mux`。译码器还可用于地址译码，其输出用作选择信号，例如选择连接到微处理器的不同 IO 设备。

6.3 编码器

编码器把独热编码的输入信号转换为二进制编码的输出信号。编码器执行的是译码器的逆操作。

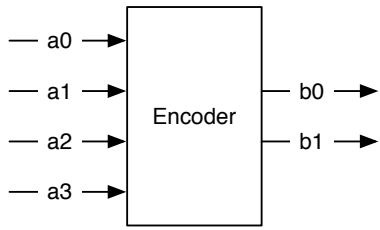


图 6.2： 4 位到 2 位编码器。

a	b
0001	00
0010	01
0100	10
1000	11
????	??

表 6.2： 4 到 2 编码器的真值表。

图 6.2 展示了一个 4 位独热输入到 2 位二进制输出的编码器，表 6.3 给出了编码功能的真值表。不过，编码器只有在输入信号为独热编码时才能按预期工作。对于其他输入值，输出是未定义的。由于无法描述输出未定义的函数，我们使用一个默认赋值来覆盖所有未定义的输入模式。

下面的 Chisel 代码先赋默认值 00，然后对合法输入值使用 switch 语句。

```
b := "b00".U
switch (a) {
  is ("b0001".U) { b := "b00".U}
  is ("b0010".U) { b := "b01".U}
  is ("b0100".U) { b := "b10".U}
  is ("b1000".U) { b := "b11".U}
}
```

6.4 练习

请描述一个组合电路，把 4 位二进制输入转换为七段数码管的编码。你可以只定义十进制数字的编码（这是七段数码管最初的用途），也可以额外定义其余位模式的编码，以便用十六进制显示单个数字的全部 16 个值。如果你有一块带七段数码管的 FPGA 开发板，请把 4 个开关或按键连接到电路输入端，并把输出连接到七段数码管。

第七章　时序逻辑构建模块

时序电路的输出不仅取决于当前输入，还取决于之前的值。由于我们关注的是同步设计（时钟驱动设计），本书所说的时序电路均指同步时序电路。[1] 要构建时序电路，需要能够存储状态的元件，也就是所谓的寄存器。

7.1　寄存器

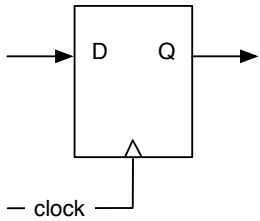


图 7.1：基于 D 触发器的寄存器。

构建时序电路的基本元件是寄存器。一个寄存器由一组 D 触发器构成。D 触发器在时钟上升沿捕获输入值，并将其保存在输出端。换句话说，寄存器在时钟上升沿用输入值更新其输出。

图 7.1 展示了寄存器的原理图符号。它有一个输入 `D` 和一个输出 `Q`。每个寄存器还有一个 `clock` 信号输入。由于这个全局时钟信号连接到同步电路中的所有寄存器，通常不在原理图中画出。方框底部的小三角形表示时钟输入，也提示我们这是一个寄存器。在后续的图中，我们将省略时钟信号。省略全局时钟信号这一点也反映在 Chisel 中：不需要显式地把某个信号连接到寄存器的时钟输入。

在 Chisel 中，带输入 `d` 和输出 `q` 的寄存器定义如下：

```
val q = RegNext(d)
```

注意，我们不需要把时钟连接到寄存器，Chisel 会隐式完成这一连接。寄存器的输入和输出可以是任意复杂类型，由向量和 Bundle 组合而成。

寄存器也可以分两步定义和使用：

```
val delayReg = Reg(UInt(4.W))

delayReg := delayIn
```

[1] 也可以用异步逻辑和反馈构建时序电路，但这是一个特定的细分领域，无法用 Chisel 表达。

首先定义寄存器并给它命名，然后把信号 `delayIn` 连接到寄存器的输入。还要注意，寄存器名称中包含字符串 `Reg`。为了便于区组合电路和时序电路，常见的做法是在名称中包含 `Reg` 标记。此外，Scala（以及 Chisel）中的名称通常采用驼峰命名法：变量名以小写字母开头，类名以大写字母开头。

寄存器还可以在复位时初始化。与 `clock` 一样，`reset` 信号在 Chisel 中是隐式的。我们把复位值（例如 0）作为参数传给寄存器构造器 `RegInit`。寄存器的输入通过 Chisel 赋值语句连接。

```
val valReg = RegInit(0.U(4.W))

valReg := inVal
```

Chisel 中默认的复位实现是同步复位。[2] 对于同步复位，D 触发器本身无需改动，只需在输入端增加一个多路选择器，在复位时的初始化值和数据值之间进行选择。

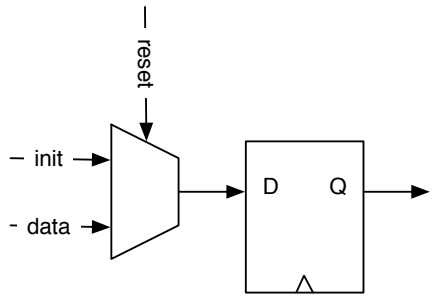


图 7.2：带同步复位的 D 触发器寄存器。

图 7.2 展示了带同步复位的寄存器原理图，其中复位信号驱动多路选择器。不过，由于同步复位非常常用，现代 FPGA 的触发器带有同步复位（和置位）输入，从而不必为多路选择器浪费 LUT 资源。

时序电路的取值会随时间变化，因此可以用一张显示信号随时间变化的图来描述其行为。这样的图称为波形图或时序图。

图 7.3 展示了一个带复位功能的寄存器的波形，并给它施加了一些输入数据。时间从左向右推进。图的上方是驱动电路的时钟。在第一个时钟周期，复位之前，寄存器的值是未定义的。第二个时钟周期中复位信号为高；在这个时钟周期的上升沿（标为 B），寄存器捕获初始值 `0`，输入 `inVal` 被忽略。下一个时钟周期中 `reset` 为 `0`，在下一个上升沿（标为 C）捕获 `inVal` 的值。此后，`reset` 保持为 `0`（这是它应有的状态），寄存器输出以一拍时钟的延迟跟随输入信号。

波形图是图形化描述电路行为的绝佳工具。特别是在多个操作并行发生、数据以流水线方

[2] 对异步复位的支持仍在开发中。

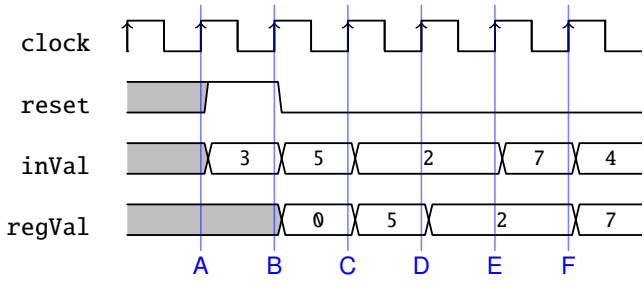


图 7.3: 带复位寄存器的波形图。

式流经电路的复杂电路中，时序图非常方便。Chisel 测试器在测试过程中也可以生成波形，可以用波形查看器显示，并用于调试。

一种典型的设计模式是带使能信号的寄存器：只有当使能信号为 `true`（高电平）时，寄存器才捕获输入，否则保持原值。与同步复位类似，使能功能可以用寄存器输入端的多路选择器实现，其中一个输入是寄存器输出的反馈。

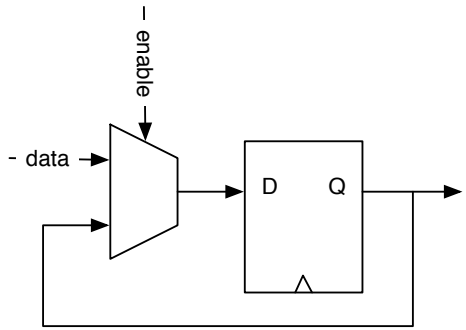


图 7.4: 带使能信号的 D 触发器寄存器。

图 7.4 展示了带使能寄存器的原理图。由于这也是一种常见的设计模式，现代 FPGA 的触发器带有专用的使能输入，不需要额外的资源。

图 7.5 给出了带使能寄存器的示例波形。大多数时候使能为高（`true`），寄存器以一拍延迟跟随输入。只有在第四个时钟周期 `enable` 为低，寄存器在上升沿 D 保持原值（5）。

带使能的寄存器可以用条件更新，以几行 Chisel 代码描述：

```
val enableReg = Reg(UInt(4.W))

when (enable) {
```

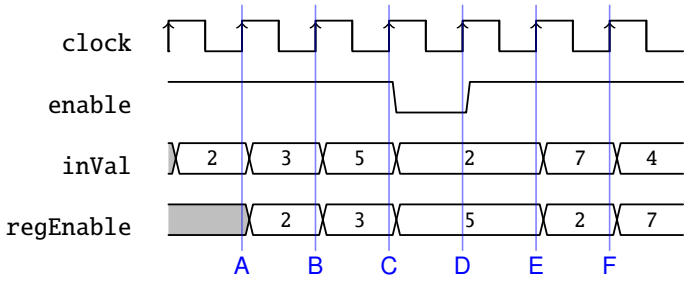


图 7.5: 带使能信号寄存器的波形图。

```
    enableReg := inVal
  }
```

带使能的寄存器同样可以带复位:

```
  val resetEnableReg = RegInit(0.U(4.W))

  when (enable) {
    resetEnableReg := inVal
  }
```

寄存器也可以作为表达式的一部分。下面的电路通过比较信号当前的值与上一个时钟周期的值，来检测信号的上升沿。

```
  val risingEdge = din & !RegNext(din)
```

现在我们已经了解了寄存器的所有基本用法，接下来用它们构建更有趣的时序电路。

7.2 计数器

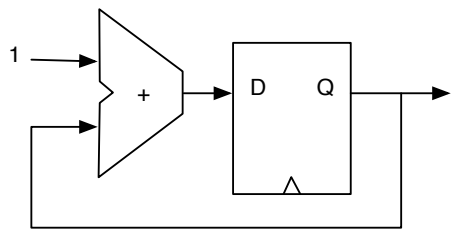


图 7.6: 加法器与寄存器构成计数器。

最基本的时序电路之一是计数器。最简单的计数器就是一个寄存器：输出连接到加法器，加法器的输出再连回寄存器输入。图 7.6 展示了这样一个自由运行计数器。

一个带 4 位寄存器的自由运行计数器从 0 计数到 15，然后回绕到 0。计数器还应能复位到一个已知值。

```
val cntReg = RegInit(0.U(4.W))

cntReg := cntReg + 1.U
```

如果想统计事件，可以用一个条件来递增计数器。

```
val cntEventsReg = RegInit(0.U(4.W))
when(event) {
  cntEventsReg := cntEventsReg + 1.U
}
```

7.2.1 向上计数与向下计数

要计数到某个值后从 0 重新开始，需要把计数器值与一个最大常量进行比较，例如使用 `when` 条件语句。

```
val cntReg = RegInit(0.U(8.W))

cntReg := cntReg + 1.U
when(cntReg === N) {
  cntReg := 0.U
}
```

也可以用多路选择器实现计数器：

```
val cntReg = RegInit(0.U(8.W))

cntReg := Mux(cntReg === N, 0.U, cntReg + 1.U)
```

如果想向下计数，可以从最大值开始计数（复位到最大值），并在计数到 0 时把计数器复位到该最大值。

```
val cntReg = RegInit(N)

cntReg := cntReg - 1.U
when(cntReg === 0.U) {
  cntReg := N
}
```

随着我们编写和使用的计数器越来越多，可以定义一个带参数的函数来生成计数器。

```
// This function returns a counter
def genCounter(n: Int) = {
```

```
    val cntReg = RegInit(0.U(8.W))
    cntReg := Mux(cntReg === n.U, 0.U, cntReg + 1.U)
    cntReg
  }

  // now we can easily create many counters
  val count10 = genCounter(10)
  val count99 = genCounter(99)
```

函数 `genCounter` 的最后一条语句就是函数的返回值，本例中返回计数寄存器 `cntReg`。

注意，在所有例子中，计数器的取值范围都是 `0` 到 `N`（包含 `N`）。如果要计数 10 个时钟周期，需要把 `N` 设为 9。把 `N` 设为 10 是差一错误的典型例子。

7.2.2 用计数器生成定时

除了统计事件，计数器还常用于生成定时。同步电路以固定频率的时钟运行，并随时钟节拍推进。数字电路中没有“时间”的概念，只有时钟节拍。如果知道时钟频率，就可以构建产生定时事件的电路，例如按照某个频率让 LED 闪烁，正如 Chisel “Hello World”示例所示。

一种常见做法是生成频率为 f_{tick} 的单周期节拍（tick），供电路使用。该节拍每隔 n 个时钟周期出现一次，其中 $n = f_{clock}/f_{tick}$，并且节拍正好持续一个时钟周期。这个节拍不用作派生时钟，而是作为电路中那些按频率 f_{tick} 逻辑工作的寄存器的使能信号。图 7.7 展示了每 3 个时钟周期产生一个节拍的例子。

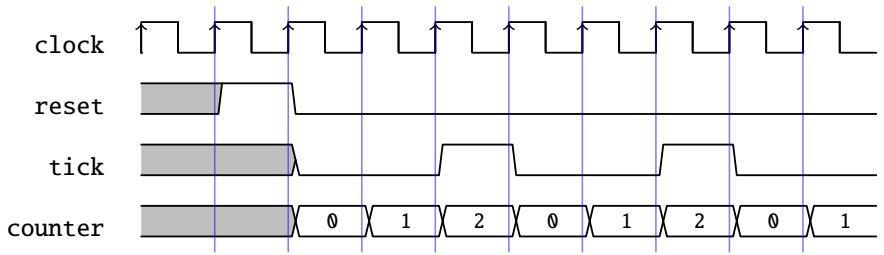


图 7.7: 生成慢频率节拍的波形图。

在下面的电路中，我们描述一个从 `0` 计数到最大值 `N - 1` 的计数器。到达最大值时，`tick` 在一个周期内为 `true`，计数器复位到 `0`。从 `0` 计数到 `N - 1`，就相当于每 `N` 个时钟周期产生一个逻辑节拍。

```
val tickCounterReg = RegInit(0.U(4.W))
val tick = tickCounterReg === (N-1).U

tickCounterReg := tickCounterReg + 1.U
when (tick) {
```

```
  tickCounterReg := 0.U
}
```

这种每 n 个时钟周期产生一个节拍的逻辑定时，可以用来推动电路中其他部分的慢速逻辑时钟。在下面的代码中，我们只是再用一个计数器，让它每 n 个时钟周期递增 1。

```
val lowFrequCntReg = RegInit(0.U(4.W))
when (tick) {
  lowFrequCntReg := lowFrequCntReg + 1.U
}
```

这种较慢的逻辑时钟的典型用途包括：让 LED 闪烁、为串行总线生成波特率、为七段数码管的动态扫描生成信号，以及对输入值进行降采样以实现按键和开关的去抖动。

虽然宽度推断应该能够确定寄存器的大小，但最好在寄存器定义时通过类型或初始值显式指定位宽。显式指定位宽可以避免一些意外，例如复位值 `0.U` 导致计数器只有 1 位宽。

7.2.3 极客计数器

我们中的许多人有时会觉得自己是个极客。例如，我们想设计一个高度优化的计数器/节拍生成器。标准计数器需要以下资源：一个寄存器、一个加法器（或减法器）和一个比较器。寄存器和加法器没有太多优化空间。向上计数时，需要与一个数（即位串）进行比较。比较器可以由针对位串中 0 的反相器和一个大与门构成。向下计数到零时，比较器是一个大或非门；在 ASIC 中，这可能比与常量比较稍微便宜一些。在 FPGA 中，逻辑由查找表实现，因此与 0 还是与 1 比较没有区别。向上计数和向下计数所需的资源相同。

不过，聪明的硬件设计师还能再耍一个花招。到目前为止，向上或向下计数都需要比较所有计数位。如果我们从 N-2 向下计数到 -1 呢？负数的最高有效位为 1，正数的最高有效位为 0。这样只需检查这一位就能检测到计数器到达 -1。这就是极客设计的计数器：

```
val MAX = (N - 2).S(8.W)
val cntReg = RegInit(MAX)
io.tick := false.B

cntReg := cntReg - 1.S
when(cntReg(7)) {
  cntReg := MAX
  io.tick := true.B
}
```

7.3 存储器

存储器可以用一组寄存器构成，在 Chisel 中就是 `Vec` 类型的 `Reg`。但这在硬件上代价很高，较大的存储结构通常使用 SRAM 构建。对于 ASIC，存储器由存储器编译器生成。FPGA

包含片上存储块，也称为块 RAM（block RAM）。这些片上存储块可以组合成更大的存储器。FPGA 中的存储器通常有一个读端口和一个写端口，或者有两个可以切换方向的端口。

FPGA（以及 ASIC）通常支持同步存储器。同步存储器在输入端带有寄存器（读地址、写地址、写数据和写使能），这意味着在读地址设置后，读数据要过一个时钟周期才可用。

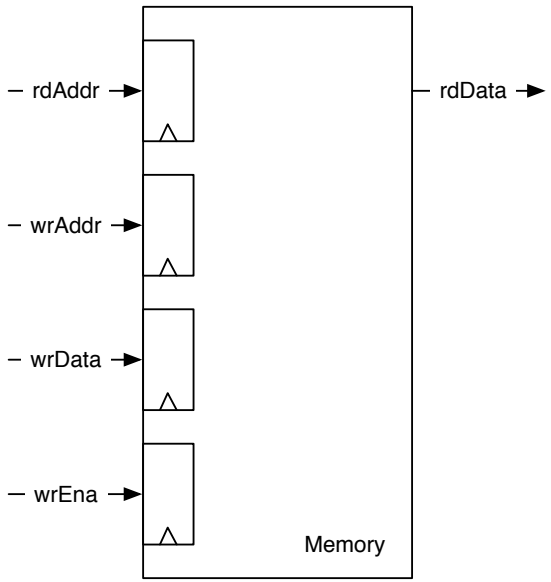


图 7.8： 同步存储器。

图 7.8 展示了这种同步存储器的原理图。该存储器是双端口的，有一个读端口和一个写端口。读端口有一个输入：读地址（`rdAddr`），以及一个输出：读数据（`rdData`）。写端口有三个输入：地址（`wrAddr`）、要写入的数据（`wrData`）和写使能（`wrEna`）。注意，所有输入在存储器内部都有寄存器，以实现同步行为。

为了支持片上存储器，Chisel 提供了存储器构造器 `SyncReadMem`。代码清单 7.1 展示了一个 `Memory` 组件，实现了 1 KB 的存储器，输入输出数据均为字节宽，并带有写使能。

```
class Memory() extends Module {
  val io = IO(new Bundle {
    val rdAddr = Input(UInt(10.W))
    val rdData = Output(UInt(8.W))
    val wrEna = Input(Bool())
    val wrData = Input(UInt(8.W))
    val wrAddr = Input(UInt(10.W))
  })
```

```
  val mem = SyncReadMem(1024, UInt(8.W))

  io.rdData := mem.read(io.rdAddr)

  when(io.wrEna) {
    mem.write(io.wrAddr, io.wrData)
  }
}
```

代码清单 7.1: 1 KB 的同步存储器。

一个有趣的问题是：如果在同一个时钟周期内，向正在读取的地址写入新值，读操作会返回什么值？我们关心的是存储器的写后立即读（read-during-write）行为。有三种可能：返回新写入的值、返回旧值，或者返回未定义值（可能是旧值和新写入数据的某些位的混合）。FPGA 支持哪种行为取决于 FPGA 类型，有时可以指定。Chisel 文档指出读数据是未定义的。

如果希望读出刚写入的值，可以构建一个前递（forwarding）电路：检测读写地址是否相同，并前递写入数据。图 7.9 展示了带前递电路的存储器。读写地址进行比较，并与写使能一起作为选择条件，在多路选择器的写数据前递路径和存储器读数据之间进行选择。写入数据经过一个寄存器延迟一个时钟周期。

代码清单 7.2 展示了包含前递电路的同步存储器 Chisel 代码。我们需要把写入数据保存到寄存器（`wrDataReg`）中，以便在下一个时钟周期使用，因为同步存储器也有一拍时钟的延迟。我们比较两个输入地址（`wrAddr` 和 `rdAddr`），并检查 `wrEna` 是否为真，作为前递条件。该条件也需要延迟一个时钟周期。最后用多路选择器在前递（写入）数据和存储器读数据之间进行选择。

```
class ForwardingMemory() extends Module {
  val io = IO(new Bundle {
    val rdAddr = Input(UInt(10.W))
    val rdData = Output(UInt(8.W))
    val wrEna = Input(Bool())
    val wrData = Input(UInt(8.W))
    val wrAddr = Input(UInt(10.W))
  })

  val mem = SyncReadMem(1024, UInt(8.W))

  val wrDataReg = RegNext(io.wrData)
  val doForwardReg = RegNext(io.wrAddr === io.rdAddr && io.wrEna)

  val memData = mem.read(io.rdAddr)

  when(io.wrEna) {
    mem.write(io.wrAddr, io.wrData)
  }

  io.rdData := Mux(doForwardReg, wrDataReg, memData)
}
```

代码清单 7.2: 带前递电路的存储器。

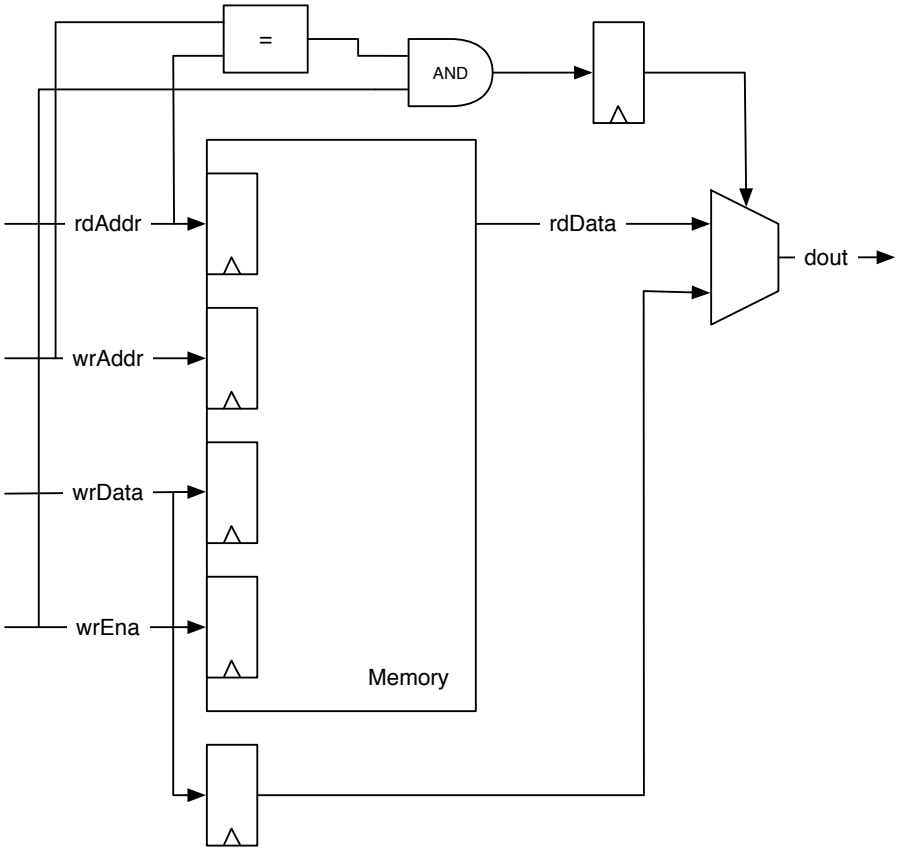


图 7.9: 带前递电路的同步存储器，具有确定的写后读行为。

Chisel 还提供了 `Mem`，它表示同步写、异步读的存储器。由于 FPGA 通常不直接提供这种存储器类型，综合工具会用触发器来实现它。因此，我们推荐使用 `SyncReadMem`。

7.4 练习

使用上一练习中的七段数码管编码器，并添加一个 4 位计数器作为输入，让显示在 `0` 到 `F` 之间变化。如果把计数器直接连接到 FPGA 开发板的时钟，你会看到 16 个数字重叠在一起（所有 7 段都会点亮）。因此需要放慢计数速度。请再创建一个计数器，每隔 500 毫秒产生一个单周期节拍信号，并把该信号用作 4 位计数器的使能信号。

第八章　有限状态机

有限状态机（FSM）是数字设计中的基本构建模块。FSM 可以描述为一组状态，以及状态之间带条件（守卫）的状态转移。FSM 有一个初始状态，在复位时设置。FSM 也称为同步时序电路。

FSM 的实现由三部分组成：(1) 保存当前状态的寄存器；(2) 根据当前状态和输入计算次态的组合逻辑；(3) 计算 FSM 输出的组合逻辑。

原则上，任何包含寄存器或其他存储状态元件的数字电路，都可以描述为单个 FSM。但这可能并不实用，例如，试着把你的笔记本电脑描述成单个 FSM。下一章将介绍如何把较小的 FSM 组合成相互通信的 FSM，从而构建更大的系统。

8.1　基本的有限状态机

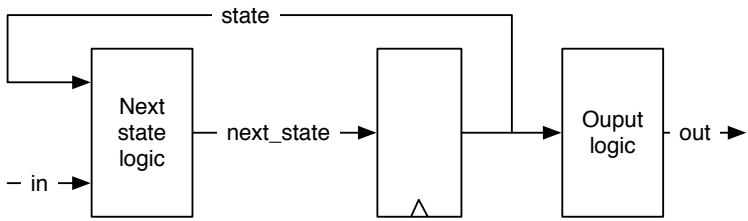


图 8.1：有限状态机（摩尔型）。

图 8.1 展示了 FSM 的原理图。寄存器保存当前状态 `state`。次态逻辑根据当前 `state` 和输入 `in` 计算次态值 `next_state`。在下一个时钟节拍，`state` 变为 `next_state`。输出逻辑计算输出 `out`。由于输出只取决于当前状态，这种状态机称为摩尔型状态机。

状态图可以直观地描述这种 FSM 的行为。在状态图中，各个状态画成圆圈，并用状态名标注。状态转移用状态之间的箭头表示。转移发生的守卫条件（或条件）标注在箭头旁。

图 8.2 展示了一个简单示例 FSM 的状态图。该 FSM 有三个状态：*green*（绿）、*orange*（橙）和 *red*（红），表示不同的告警级别。FSM 从 *green* 级别开始。发生一次坏事件（bad event）时，告警级别切换到 *orange*。再发生一次坏事件，告警级别切换到 *red*。在这种情况下，我们希望响铃；*ring bell* 是这个 FSM 唯一的输出。我们把该输出加到 *red* 状态上。告警可以通过 *clear* 信号复位。

状态图虽然直观、能让人快速理解 FSM 的功能，但状态表可能更容易写下来。表 8.1 给出了告警 FSM 的状态表。表中列出当前状态、输入值、对应的次态，以及当前状态的输出值。原则上，需要为所有可能状态列出所有可能的输入组合，该表将有 $3 \times 4 = 12$ 行。我们简化了表格：发生坏事件时，*clear* 输入是无关项。也就是说，坏事件的优先级高于 *clear*。输出列有

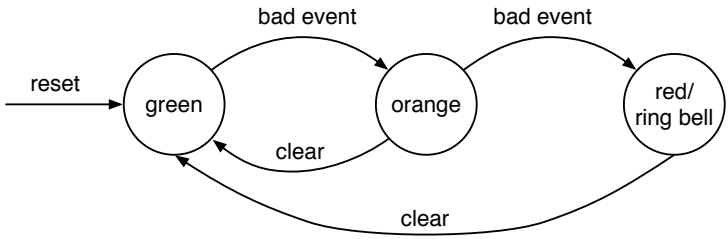


图 8.2: 告警 FSM 的状态图。

一些重复。如果 FSM 更大和/或输出更多，可以把表拆成两张：一张用于次态逻辑，一张用于输出逻辑。

表 8.1: 告警 FSM 的状态表。

	输入			
状态	坏事件	清除	次态	响铃
green	0	0	green	0
green	1	-	orange	0
orange	0	0	orange	0
orange	1	-	red	0
orange	0	1	green	0
red	0	0	red	1
red	0	1	green	1

完成告警级别 FSM 的设计后，我们终于可以用 Chisel 编写它了。代码清单 8.1 展示了告警 FSM 的 Chisel 代码。注意，FSM 的输入和输出使用 Chisel 的 `Bool` 类型。要使用 `Enum` 和 `switch` 控制指令，需要导入 `chisel3.util._`。

```
import chisel3._
import chisel3.util._

class SimpleFsm extends Module {
  val io = IO(new Bundle{
    val badEvent = Input(Bool())
    val clear = Input(Bool())
    val ringBell = Output(Bool())
  })

  // The three states
```

```
  val green :: orange :: red :: Nil = Enum(3)

  // The state register
  val stateReg = RegInit(green)

  // Next state logic
  switch (stateReg) {
    is (green) {
      when(io.badEvent) {
        stateReg := orange
      }
    }
    is (orange) {
      when(io.badEvent) {
        stateReg := red
      } .elsewhen(io.clear) {
        stateReg := green
      }
    }
    is (red) {
      when (io.clear) {
        stateReg := green
      }
    }
  }

  // Output logic
  io.ringBell := stateReg === red
}
```

代码清单 8.1: 告警 FSM 的 Chisel 代码。

这个简单 FSM 的完整 Chisel 代码可以放在一页内。我们逐部分来看。FSM 有两个输入和一个输出信号，都放在一个 Chisel `Bundle` 中:

```
  val io = IO(new Bundle{
    val badEvent = Input(Bool())
    val clear = Input(Bool())
    val ringBell = Output(Bool())
  })
```

最优的状态编码需要花不少功夫。两种常见选择是二进制编码和独热编码。不过，我们把这些底层决策留给综合工具，着眼于代码的可读性。[1] 因此，我们使用带符号名称的枚举类型来表示状态:

```
  val green :: orange :: red :: Nil = Enum(3)
```

各个状态值用列表描述，列表元素用`::`运算符连接，`Nil` 表示列表结尾。一个 `Enum` 实例被赋

[1] 当前版本的 Chisel 中，`Enum` 类型以二进制编码表示状态。如果需要其他编码，例如独热编码，可以为状态名定义 Chisel 常量。

值给状态列表。保存状态的寄存器以 *green* 状态作为复位值：

```
val stateReg = RegInit(green)
```

FSM 的核心在于次态逻辑。我们对状态寄存器使用 Chisel 的 switch 语句，覆盖所有状态。在每个 `is` 分支中，根据输入编写次态逻辑，为状态寄存器赋新值：

```
switch (stateReg) {
  is (green) {
    when(io.badEvent) {
      stateReg := orange
    }
  }
  is (orange) {
    when(io.badEvent) {
      stateReg := red
    } .elsewhen(io.clear) {
      stateReg := green
    }
  }
  is (red) {
    when (io.clear) {
      stateReg := green
    }
  }
}
```

最后，当状态为 *red* 时，把响铃输出置为真。

```
io.ringBell := stateReg === red
```

注意，我们没有像 Verilog 或 VHDL 那样为寄存器输入引入一个 `next_state` 信号。Verilog 和 VHDL 中的寄存器用特殊语法描述，不能在组合逻辑块中赋值（和重新赋值）。因此需要引入一个在组合逻辑块中计算得到的额外信号，再连接到寄存器输入。在 Chisel 中，寄存器是基本类型，可以在组合逻辑块中自由使用。

8.2 用米利型 FSM 加快输出

在摩尔型 FSM 中，输出只取决于当前状态。这意味着输入的变化最早在下一个时钟周期才能反映到输出上。如果希望立即观察到变化，就需要一条从输入到输出的组合路径。让我们看一个最小的例子：边沿检测电路。前面已经见过这个 Chisel 一行代码：

```
val risingEdge = din & !RegNext(din)
```

图 8.3 展示了上升沿检测器的原理图。当当前输入为 1、上一时钟周期的输入为 0 时，输出在一个时钟周期内变为 1。状态寄存器只是一个 D 触发器，其次态就是输入。也可以把它看作一拍时钟的延迟元件。输出逻辑比较当前输入与当前状态。

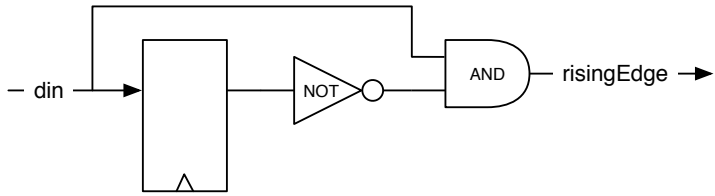


图 8.3： 上升沿检测器（米利型 FSM）。

当输出也取决于输入，即 FSM 的输入和输出之间存在组合路径时，这种状态机称为米利型状态机。

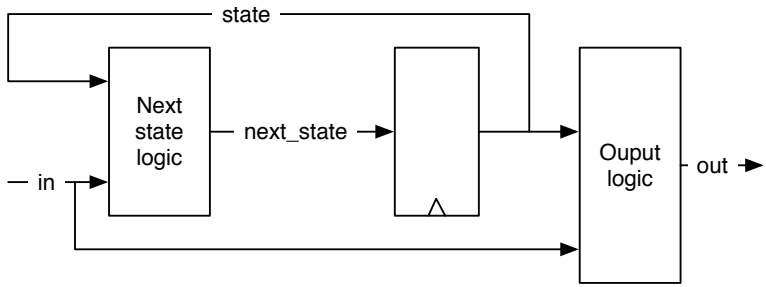


图 8.4： 米利型有限状态机。

图 8.4 展示了米利型 FSM 的原理图。与摩尔型 FSM 类似，寄存器保存当前 `state`，次态逻辑根据当前 `state` 和输入 `in` 计算次态值 `next_state`。在下一个时钟节拍，`state` 变为 `next_state`。输出逻辑根据当前状态和 FSM 的输入计算输出 `out`。

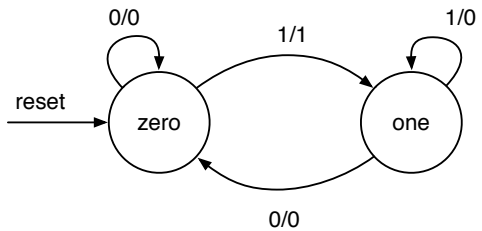


图 8.5： 上升沿检测器作为米利型 FSM 的状态图。

图 8.5 展示了边沿检测器米利型 FSM 的状态图。由于状态寄存器只包含一个 D 触发器，

只可能有两个状态，本例中命名为 zero 和 one。由于米利型 FSM 的输出不仅取决于状态，还取决于输入，输出不能写在状态圆圈里。取而代之，状态之间的转移标注为输入值（条件）和输出（写在斜杠之后）。还要注意，我们画出了自转移：例如在状态 zero 中，当输入为 0 时，FSM 保持在状态 zero，输出为 0。上升沿 FSM 只在从状态 zero 转移到状态 one 时产生 1 输出。在状态 one（表示输入现在为 1）时，输出为 0。对于输入的每个上升沿，我们只需要一个（时钟周期的）脉冲。

```
import chisel3._
import chisel3.util._

class RisingFsm extends Module {
  val io = IO(new Bundle{
    val din = Input(Bool())
    val risingEdge = Output(Bool())
  })

  // The two states
  val zero :: one :: Nil = Enum(2)

  // The state register
  val stateReg = RegInit(zero)

  // default value for output
  io.risingEdge := false.B

  // Next state and output logic
  switch (stateReg) {
    is(zero) {
      when(io.din) {
        stateReg := one
        io.risingEdge := true.B
      }
    }
    is(one) {
      when(!io.din) {
        stateReg := zero
      }
    }
  }
}
```

代码清单 8.2: 用米利型 FSM 检测上升沿。

代码清单 8.2 展示了用米利型状态机检测上升沿的 Chisel 代码。与前一个例子一样，单比特输入和输出使用 Chisel 的 Bool 类型。输出逻辑现在是次态逻辑的一部分：在从 zero 转移到 one 时，输出被置为 true.B；其他情况下，输出的默认赋值（false.B）生效。

有人可能会问：对于边沿检测电路，完整的 FSM 是否是最佳方案？尤其是我们已经见过功能相同的 Chisel 一行代码。两者硬件开销相近：都需要一个 D 触发器保存状态。FSM 的组合逻辑可能稍复杂一些，因为状态变化取决于当前状态和输入值。对于这个功能，一行代码更

易写、更易读，而可读性更重要。因此，一行代码是首选的解决方案。

我们使用这个例子来展示尽可能小的米利型 FSM 之一。对于三个及以上状态的更复杂电路，才应该使用 FSM。

8.3 摩尔型与米利型对比

为了展示摩尔型和米利型 FSM 的区别，我们用摩尔型 FSM 重新实现边沿检测。

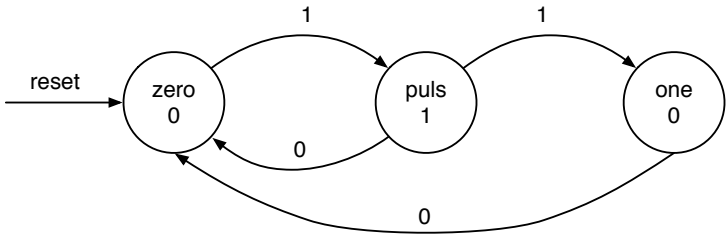


图 8.6: 上升沿检测器作为摩尔型 FSM 的状态图。

图 8.6 展示了用摩尔型 FSM 检测上升沿的状态图。首先要注意，摩尔型 FSM 需要三个状态，而米利型版本只需要两个。需要状态 `puls` 来产生单周期脉冲。FSM 只在状态 `puls` 停留一个时钟周期，然后要么回到起始状态 `zero`，要么进入 `one` 状态，等待输入再次变为 0。我们在状态转移箭头上标出输入条件，在表示状态的圆圈内标出 FSM 输出。

```
import chisel3._
import chisel3.util._

class RisingMooreFsm extends Module {
  val io = IO(new Bundle{
    val din = Input(Bool())
    val risingEdge = Output(Bool())
  })

  // The three states
  val zero :: puls :: one :: Nil = Enum(3)

  // The state register
  val stateReg = RegInit(zero)

  // Next state logic
  switch (stateReg) {
    is(zero) {
      when(io.din) {
        stateReg := puls
      }
    }
```

```
    is(puls) {
      when(io.din) {
        stateReg := one
      } .otherwise {
        stateReg := zero
      }
    }
    is(one) {
      when(!io.din) {
        stateReg := zero
      }
    }
  }

  // Output logic
  io.risingEdge := stateReg === puls
}
```

代码清单 8.3: 用摩尔型 FSM 检测上升沿。

代码清单 8.3 展示了上升沿检测电路的摩尔型版本。它使用的 D 触发器数量是米利型或直接编码版本的两倍，因此次态逻辑也比米利型或直接编码版本更大。

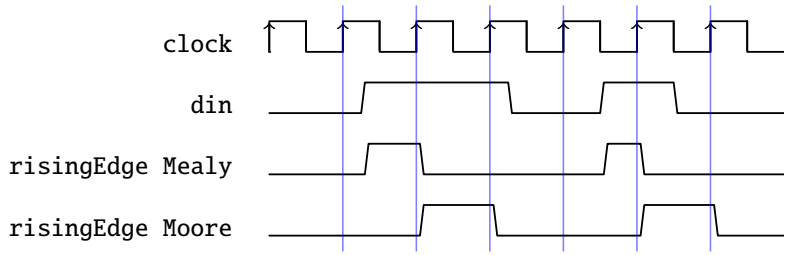


图 8.7: 上升沿检测的米利型和摩尔型 FSM 波形。

图 8.7 展示了上升沿检测 FSM 的米利型和摩尔型波形。可以看到，米利型输出紧跟输入的上升沿，而摩尔型输出在时钟节拍之后才升高。还可以看到，摩尔型输出持续一个时钟周期，而米利型输出通常小于一个时钟周期。

从上面的例子看，人们容易认为米利型 FSM 是更好的 FSM，因为它们需要的状态（以及逻辑）更少，反应也比摩尔型 FSM 快。然而，米利型状态机内部的组合路径在较大设计中会带来麻烦。第一，在多个 FSM 相互通信时（见下一章），这条组合路径可能变得很长。第二，如果相互通信的 FSM 构成环路，就会形成组合环路，这是同步设计中的错误。由于摩尔型 FSM 的状态寄存器在组合路径上提供了切分点，上述问题在相互通信的摩尔型 FSM 中都不存在。

总之，摩尔型 FSM 更适合组合成相互通信的状态机，比米利型 FSM 更健壮。只有在同一周期内做出反应至关重要时，才应使用米利型 FSM。像上升沿检测这样的小电路（实际上就

是米利型状态机）也没有问题。

8.4 练习

本章已经给出了许多非常小的 FSM 示例。现在该编写一些真正的 FSM 代码了。选择一个稍复杂一些的例子，实现相应的 FSM，并编写测试平台。

FSM 的一个经典例子是交通灯控制器（见 [3, 第 14.3 节]）。交通灯控制器必须保证：在从红灯切换到绿灯时，中间要有一个阶段，让十字路口的两个方向都处于禁止通行状态（红灯和橙灯）。为了让这个例子更有趣一些，考虑一条主干道：支路上有两个车辆检测器（位于进入十字路口的两个入口）。只有检测到车辆时，才把支路切换为绿灯；之后再把主干道切回绿灯。

第九章　相互通信的状态机

一个问题往往过于复杂，难以用单个 FSM 描述。这时可以把问题划分为两个或更多更小、更简单的 FSM。这些 FSM 通过信号相互通信：一个 FSM 的输出是另一个 FSM 的输入，后者观察前者的输出。把一个大型 FSM 拆分成若干更简单的 FSM，称为 FSM 分解（factoring）。不过，相互通信的 FSM 通常直接根据规格说明来设计，因为单个 FSM 往往会大到难以实现。

9.1　灯光闪烁器示例

为了讨论相互通信的 FSM，我们使用文献 [3, 第 17 章] 中的灯光闪烁器例子。灯光闪烁器有一个输入 `start` 和一个输出 `light`。其规格说明如下：

- 当 `start` 在一个时钟周期内为高时，闪烁序列开始；
- 序列闪烁三次；
- 每次闪烁时，`light` 点亮六个时钟周期，两次闪烁之间 `light` 熄灭四个时钟周期；
- 序列结束后，FSM 让 `light` 熄灭，并等待下一次启动。

直接实现的 FSM[1] 有 27 个状态：一个等待输入的初始状态，三个点亮阶段需要 3×6 个状态，熄灭阶段需要 2×4 个状态。我们不再展示灯光闪烁器这种简单直接实现的代码。

更优雅的解法是把这个大 FSM 分解为两个较小的 FSM：主 FSM 实现闪烁逻辑，定时器 FSM 实现等待。图 9.1 展示了这两个 FSM 的组合。

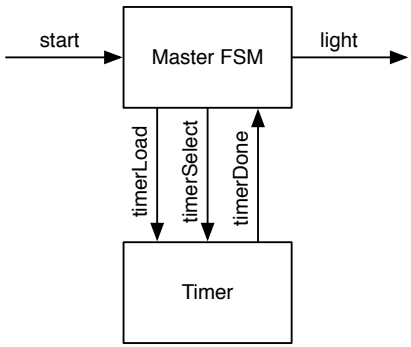


图 9.1：拆分为主 FSM 和定时器 FSM 的灯光闪烁器。

定时器 FSM 向下计数 6 或 4 个时钟周期，以产生所需的定时。定时器的规格说明如下：

[1] 状态图见文献 [3, 第 376 页]。

- 当 timerLoad 有效时，定时器把值装入递减计数器，与当前状态无关；
- timerSelect 在装载值 5 或 3 之间进行选择；
- 当计数器完成倒计数后，timerDone 置位并保持；
- 其他情况下，定时器继续递减计数。

下面的代码展示了灯光闪烁器的定时器 FSM：

```
val timerReg = RegInit(0.U)
timerDone := timerReg === 0.U

// Timer FSM (down counter)
when(!timerDone) {
  timerReg := timerReg - 1.U
}
when (timerLoad) {
  when (timerSelect) {
    timerReg := 5.U
  } .otherwise {
    timerReg := 3.U
  }
}
```

代码清单 9.1 展示了主 FSM。

```
val off :: flash1 :: space1 :: flash2 :: space2 :: flash3 :: Nil = Enum(6)
val stateReg = RegInit(off)

val light = WireInit(false.B) // FSM output

// Timer connection
val timerLoad = WireInit(false.B) // start timer with a load
val timerSelect = WireInit(true.B) // select 6 or 4 cycles
val timerDone = Wire(Bool())

timerLoad := timerDone

// Master FSM
switch(stateReg) {
  is(off) {
    timerLoad := true.B
    timerSelect := true.B
    when (start) { stateReg := flash1 }
  }
  is (flash1) {
    timerSelect := false.B
    light := true.B
    when (timerDone) { stateReg := space1 }
  }
  is (space1) {
    when (timerDone) { stateReg := flash2 }
  }
```

```
    is (flash2) {
      timerSelect := false.B
      light := true.B
      when (timerDone) { stateReg := space2 }
    }
    is (space2) {
      when (timerDone) { stateReg := flash3 }
    }
    is (flash3) {
      timerSelect := false.B
      light := true.B
      when (timerDone) { stateReg := off }
    }
  }
```

代码清单 9.1: 灯光闪烁器的主 FSM。

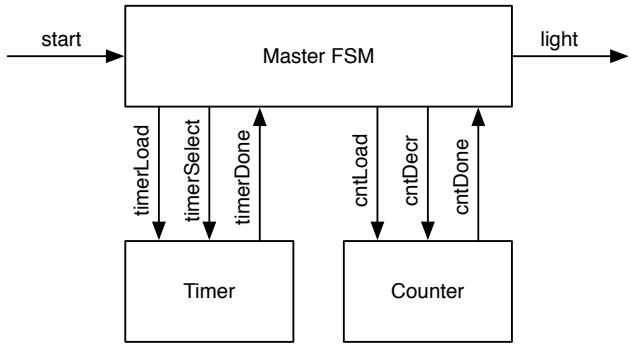


图 9.2: 拆分为主 FSM、定时器 FSM 和计数器 FSM 的灯光闪烁器。

主 FSM 加定时器的方案中，主 FSM 的代码仍有冗余。状态 `flash1`、`flash2` 和 `flash3` 执行相同的功能，状态 `space1` 和 `space2` 也是如此。我们可以把剩余闪烁次数提取到一个额外的计数器中。这样主 FSM 就缩减为三个状态：`off`、`flash` 和 `space`。

图 9.2 展示了由一个主 FSM 和两个计数 FSM 组成的设计：一个 FSM 对点亮和熄灭间隔的时钟周期计数，另一个 FSM 对剩余闪烁次数计数。

下面的代码展示了递减计数器 FSM：

```
val cntReg = RegInit(0.U)
cntDone := cntReg === 0.U

// Down counter FSM
when(cntLoad) { cntReg := 2.U }
when(cntDecr) { cntReg := cntReg - 1.U }
```

注意，闪烁 3 次时计数器装载的初值是 2，因为它统计的是剩余闪烁次数，并在定时器完成时于 space 状态递减。代码清单 9.2 展示了两次重构后的灯光闪烁器主 FSM。

```
val off :: flash :: space :: Nil = Enum(3)
val stateReg = RegInit(off)

val light = WireInit(false.B) // FSM output

// Timer connection
val timerLoad = WireInit(false.B) // start timer with a load
val timerSelect = WireInit(true.B) // select 6 or 4 cycles
val timerDone = Wire(Bool())
// Counter connection
val cntLoad = WireInit(false.B)
val cntDecr = WireInit(false.B)
val cntDone = Wire(Bool())

timerLoad := timerDone

switch(stateReg) {
  is(off) {
    timerLoad := true.B
    timerSelect := true.B
    cntLoad := true.B
    when (start) { stateReg := flash }
  }
  is (flash) {
    timerSelect := false.B
    light := true.B
    when (timerDone & !cntDone) { stateReg := space }
    when (timerDone & cntDone) { stateReg := off }
  }
  is (space) {
    cntDecr := timerDone
    when (timerDone) { stateReg := flash }
  }
}
```

代码清单 9.2: 两次重构后的灯光闪烁器主 FSM。

除了主 FSM 缩减到只有三个状态，当前的方案也更易于配置。如果要改变点亮或熄灭间隔的长度，或者改变闪烁次数，都不需要修改任何 FSM。

本节探讨了相互通信的电路，特别是只交换控制信号的 FSM。不过，电路也可以交换数据。为了协调数据交换，我们使用握手信号。下一节介绍用于单向数据流控制的 ready-valid 接口。

9.2 带数据通路的状态机

相互通信状态机的一个典型例子是状态机与数据通路的组合。这种组合通常称为带数据通路的状态机（FSMD）。状态机控制数据通路，数据通路执行计算。FSM 的输入来自环境和数

据通路；来自环境的数据送入数据通路，数据输出则来自数据通路。图 9.3 展示了 FSM 与数据通路组合的一个例子。

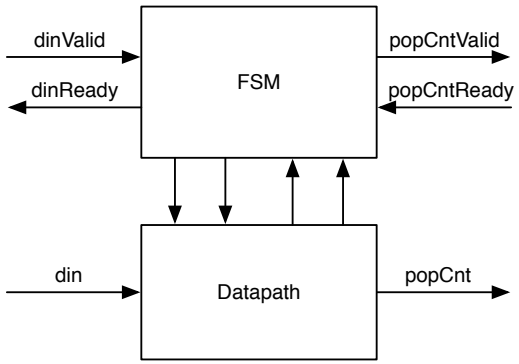


图 9.3： 带数据通路的状态机。

9.2.1 位计数示例

图 9.3 中的 FSMD 是一个计算位计数（popcount）的例子，位计数也称为汉明重量。汉明重量是不同于零符号的符号个数。对于二进制串来说，就是 '1' 的个数。

位计数单元包含数据输入 `din` 和结果输出 `popCount`，二者都连接到数据通路。输入和输出都使用 ready-valid 握手。有数据可用时，valid 置位；接收方能够接收数据时，ready 置位。两个信号都置位时，传输发生。握手信号连接到 FSM。FSM 向数据通路发送控制信号，并从数据通路接收状态信号。

下一步可以设计 FSM，先从状态图开始，如图 9.4 所示。FSM 从 `Idle` 状态开始，等待输入。数据到达（由 valid 信号表示）后，FSM 进入 `Load` 状态，装入移位寄存器。然后 FSM 进入 `Count` 状态，在这里顺序统计 '1' 的个数。我们使用一个移位寄存器、一个加法器、一个累加器寄存器和一个递减计数器完成计算。递减计数器到零时，计算完成，FSM 进入 `Done` 状态。在 `Done` 状态，FSM 通过置位 valid 信号表示位计数值已可供读取。收到接收方的 ready 信号后，FSM 返回 `Idle` 状态，准备计算下一个位计数。

代码清单 9.3 的顶层组件实例化 FSM 和数据通路组件，并用批量连接将它们连接起来。

```
class PopCount extends Module {
  val io = IO(new Bundle {
    val dinValid = Input(Bool())
    val dinReady = Output(Bool())
    val din = Input(UInt(8.W))
    val popCntValid = Output(Bool())
    val popCntReady = Input(Bool())
```

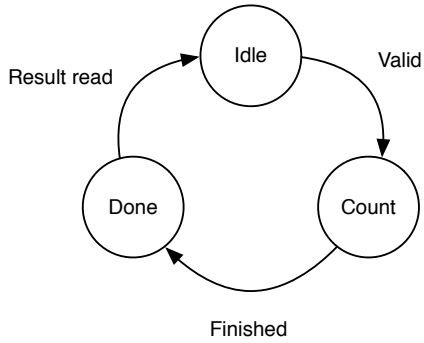


图 9.4: 位计数 FSM 的状态图。

```
    val popCnt = Output(UInt(4.W))
  })


  val fsm = Module(new PopCountFSM)
  val data = Module(new PopCountDataPath)

  fsm.io.dinValid := io.dinValid
  io.dinReady := fsm.io.dinReady
  io.popCntValid := fsm.io.popCntValid
  fsm.io.popCntReady := io.popCntReady

  data.io.din := io.din
  io.popCnt := data.io.popCnt
  data.io.load := fsm.io.load
  fsm.io.done := data.io.done
}
```

代码清单 9.3: 位计数电路的顶层。

图 9.5 展示了位计数电路的数据通路。数据装入 `shf` 寄存器，同时 `cnt` 寄存器复位为 0。为了统计 '1' 的个数，每个时钟周期把 `shf` 寄存器右移一位，并把最低有效位加到 `cnt`。图中未画出的一个计数器负责递减计数，直到所有位都移过最低有效位。计数器到零时，位计数完成。FSM 进入 `Done` 状态，并通过置位 `popCntValid` 表示结果有效。当结果被读取（由接收方置位 `popCntReady`）后，FSM 返回 `Idle`。

收到 `load` 信号时，输入装入 `regData` 寄存器，`regPopCount` 寄存器复位为 0，计数器寄存器 `regCount` 设为需要移位的次数。

其他情况下，`regData` 寄存器右移，其最低有效位加到 `regPopCount` 寄存器，计数器递减直到为 0。计数器为 0 时，输出即为位计数值。代码清单 9.4 展示了位计数电路数据通路的 Chisel

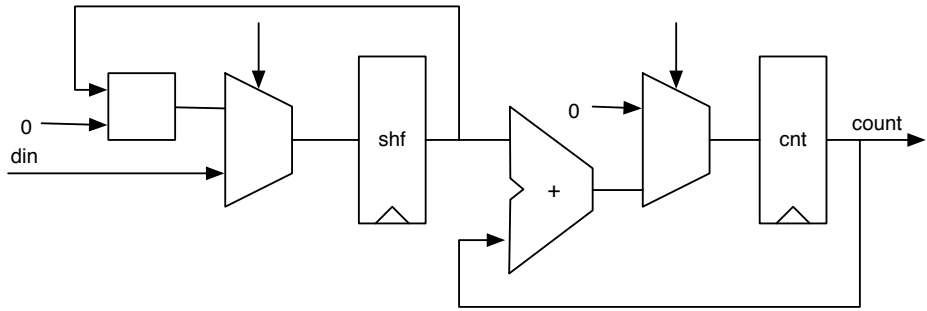


图 9.5: 位计数电路的数据通路。

代码。

```
class PopCountDataPath extends Module {
  val io = IO(new Bundle {
    val din = Input(UInt(8.W))
    val load = Input(Bool())
    val popCnt = Output(UInt(4.W))
    val done = Output(Bool())
  })

  val dataReg = RegInit(0.U(8.W))
  val popCntReg = RegInit(0.U(8.W))
  val counterReg= RegInit(0.U(4.W))

  dataReg := 0.U ## dataReg(7, 1)
  popCntReg := popCntReg + dataReg(0)

  val done = counterReg === 0.U
  when (!done) {
    counterReg := counterReg - 1.U
  }

  when(io.load) {
    dataReg := io.din
    popCntReg := 0.U
    counterReg := 8.U
  }

  // debug output
  printf("%x %d\n", dataReg, popCntReg)

  io.popCnt := popCntReg
  io.done := done
}
```

代码清单 9.4: 位计数电路的数据通路。

FSM 从 idle 状态开始。收到输入数据的 valid 信号（dinValid）后，进入 count 状态，等待数据通路完成计数。位计数结果有效时，FSM 进入 done 状态，等待结果被读取（由 popCntReady 表示）。代码清单 9.5 展示了 FSM 的代码。

```
class PopCountFSM extends Module {
  val io = IO(new Bundle {
    val dinValid = Input(Bool())
    val dinReady = Output(Bool())
    val popCntValid = Output(Bool())
    val popCntReady = Input(Bool())
    val load = Output(Bool())
    val done = Input(Bool())
  })

  val idle :: count :: done :: Nil = Enum(3)
  val stateReg = RegInit(idle)

  io.load := false.B

  io.dinReady := false.B
  io.popCntValid := false.B

  switch(stateReg) {
    is(idle) {
      io.dinReady := true.B
      when(io.dinValid) {
        io.load := true.B
        stateReg := count
      }
    }
    is(count) {
      when(io.done) {
        stateReg := done
      }
    }
    is(done) {
      io.popCntValid := true.B
      when(io.popCntReady) {
        stateReg := idle
      }
    }
  }
}
```

代码清单 9.5： 位计数电路的 FSM。

9.3 Ready-Valid 接口

子系统之间的通信可以抽象为数据移动和用于流控的握手。在位计数示例中，我们已经看到了用 valid 和 ready 信号对输入、输出数据进行握手的接口。

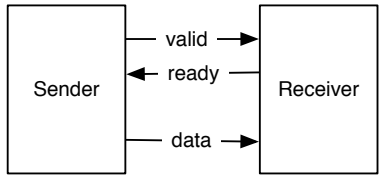


图 9.6： ready-valid 流控。

ready-valid 接口 [3, 第 480 页] 是一种简单的流控接口：发送方有 `data` 和 `valid` 信号，接收方有 `ready` 信号（见图 9.6）。发送方在 `data` 可用时置位 `valid`，接收方在准备好接收一个字的数据时置位 `ready`。当 `valid` 和 `ready` 两个信号都置位时，数据传输发生；任一信号未置位，则不进行传输。

为了使该接口可以组合，`ready` 和 `valid` 都不允许组合地依赖于对方信号。

第十章　硬件生成器

Chisel 的强大之处在于可以编写所谓的硬件生成器。使用 VHDL 和 Verilog 等较老的硬件描述语言时，我们通常要用另一种语言，例如 Java 或 Python 来生成硬件。作者本人就经常编写小型 Java 程序来生成 VHDL 表格。在 Chisel 中，构建硬件时可以使用 Scala（以及 Java 库）的全部能力。因此，我们可以用同一种语言编写硬件生成器，并把它作为 Chisel 电路生成过程的一部分来执行。

10.1　使用参数进行配置

Chisel 组件和函数可以通过参数进行配置。参数可以简单到只是一个整数常量，也可以是 Chisel 硬件类型。

10.1.1　简单参数

参数化电路的基本方法是用位宽作为参数。参数可以作为实参传给 Chisel 模块的构造器。下面的例子演示了一个可配置位宽的加法器模块。位宽 `n` 是组件的一个参数（Scala 类型 `Int`），通过构造器传入，并可在 IO Bundle 中使用。

```
class ParamAdder(n: Int) extends Module {
  val io = IO(new Bundle{
    val a = Input(UInt(n.W))
    val b = Input(UInt(n.W))
    val c = Output(UInt(n.W))
  })

  io.c := io.a + io.b
}
```

可以按如下方式创建参数化版本的加法器：

```
  val add8 = Module(new ParamAdder(8))
  val add16 = Module(new ParamAdder(16))
```

10.1.2　带类型参数的函数

把位宽作为配置参数只是硬件生成器的起点。一种非常灵活的配置方式是使用类型。这一特性使 Chisel 能够提供一个可接受任意类型的多路选择器（`Mux`）。为了展示如何用类型进行配置，我们构建一个接受任意类型的多路选择器。下面的函数定义了这个多路选择器：

```
  def myMux[T <: Data](sel: Bool, tPath: T, fPath: T): T = {

    val ret = WireInit(fPath)
    when (sel) {
```

```
    ret := tPath
  }
  ret
}
```

Chisel 允许函数对类型进行参数化，本例中是 Chisel 类型。方括号中的表达式 `[T <: Data]` 定义了一个类型参数 `T`，`T` 是 `Data` 或 `Data` 的子类。`Data` 是 Chisel 类型系统的根。

我们的多路选择器函数有三个参数：布尔条件、真值路径参数和假值路径参数。两个路径参数的类型都是 `T`，具体类型在调用函数时给出。函数本身很直接：我们定义一个默认值为 `fPath` 的 Wire，当条件为真时把值改为 `tPath`。这段逻辑就是经典的多路选择器功能。函数最后返回多路选择器硬件。

可以把多路选择器函数用于 `UInt` 等简单类型：

```
val resA = myMux(selA, 5.U, 10.U)
```

多路选择器两条路径的类型必须相同。下面错误地使用多路选择器会导致运行时错误：

```
val resErr = myMux(selA, 5.U, 10.S)
```

我们定义一个包含两个字段的 `Bundle` 类型：

```
class ComplexIO extends Bundle {
  val d = UInt(10.W)
  val b = Bool()
}
```

可以先创建 `Wire`，再设置各子字段，从而定义 `Bundle` 常量。然后就可以用这个复杂类型使用参数化多路选择器。

```
val tVal = Wire(new ComplexIO)
tVal.b := true.B
tVal.d := 42.U
val fVal = Wire(new ComplexIO)
fVal.b := false.B
fVal.d := 13.U

// The mulitplexer with a complex type
val resB = myMux(selB, tVal, fVal)
```

在函数最初的设计中，我们使用 `WireInit` 创建一个带默认值的 `T` 类型 Wire。如果只需要创建一个 Chisel 类型、不使用默认值，可以用 `fPath.cloneType` 获得该 Chisel 类型。下面的函数展示了多路选择器的另一种写法。

```
def myMuxAlt[T <: Data](sel: Bool, tPath: T, fPath: T): T = {

  val ret = Wire(fPath.cloneType)
  ret := fPath
  when (sel) {
```

```
      ret := tPath
    }
    ret
  }
```

10.1.3 带类型参数的模块

模块也可以用 Chisel 类型进行参数化。假设我们要设计一个片上网络，在不同处理核之间传输数据。但我们不想把数据格式硬编码到路由器接口中，而是想对它进行参数化。与函数的类型参数类似，我们给 Module 构造器增加一个类型参数 `T`，还需要一个该类型的构造器参数。此外，在这个例子中，我们还让路由器的端口数量可配置。

```
class NocRouter[T <: Data](dt: T, n: Int) extends Module {
  val io =IO(new Bundle {
    val inPort = Input(Vec(n, dt))
    val address = Input(Vec(n, UInt(8.W)))
    val outPort = Output(Vec(n, dt))
  })

  // Route the payload according to the address
  // ...
```

要使用这个路由器，首先需要定义要路由的数据类型，例如一个 Chisel `Bundle`：

```
class Payload extends Bundle {
  val data = UInt(16.W)
  val flag = Bool()
}
```

然后创建路由器：把用户定义的 Bundle 实例和端口数传给路由器构造器：

```
  val router = Module(new NocRouter(new Payload, 2))
```

10.1.4 参数化 Bundle

在路由器的例子中，我们为路由器输入使用了两个不同的字段向量：一个用于地址，另一个用于参数化的数据。更优雅的方案是让 `Bundle` 本身参数化，类似这样：

```
class Port[T <: Data](dt: T) extends Bundle {
  val address = UInt(8.W)
  val data = dt.cloneType
}
```

这个 `Bundle` 有一个类型为 `T` 的参数，`T` 是 Chisel `Data` 类型的子类。在 Bundle 内，通过对参数调用 `cloneType` 定义字段 `data`。然而，使用构造器参数时，该参数会成为类的公有字段。

当 Chisel 需要克隆 Bundle 的类型时（例如在 Vec 中使用它），这个公有字段会造成干扰。解决方法（变通方案）是把参数字段设为私有：

```
class Port[T <: Data](private val dt: T) extends Bundle {
  val address = UInt(8.W)
  val data = dt.cloneType
}
```

有了这个新的 Bundle，就可以定义路由器端口：

```
class NocRouter2[T <: Data](dt: T, n: Int) extends Module {
  val io =IO(new Bundle {
    val inPort = Input(Vec(n, dt))
    val outPort = Output(Vec(n, dt))
  })

  // Route the payload according to the address
  // ...
```

并用以 Payload 为参数的 Port 实例化路由器：

```
  val router = Module(new NocRouter2(new Port(new Payload), 2))
```

10.2 生成组合逻辑

在 Chisel 中，可以很容易地用 Scala Array 创建 Chisel Vec 作为逻辑表来生成逻辑。我们可能在文件中有一些数据，可以在硬件生成时读入，作为逻辑表。代码清单 10.1 展示了如何使用 Scala 标准库中的 Source 类读取文件 “data.txt”，该文件包含以文本形式表示的整数常量。

下面这个可能略显吓人的表达式：

```
  val table = VecInit(array.map(_.U(8.W)))
```

Scala Array 可以隐式转换为序列（Seq），序列支持映射函数 map。map 对序列中的每个元素调用一个函数，并返回由函数返回值组成的序列。我们的函数 _.U(8.W) 把 Scala 数组中的每个 Int 值表示为 _，并执行从 Scala Int 值到 8 位 Chisel UInt 字面量的转换。Chisel 对象 VecInit 从 Chisel 类型的序列 Seq 创建 Chisel Vec。

```
import chisel3._
import scala.io.Source

class FileReader extends Module {
  val io = IO(new Bundle {
    val address = Input(UInt(8.W))
    val data = Output(UInt(8.W))
  })

  val array = new Array[Int](256)
```

```
  var idx = 0

  // read the data into a Scala array
  val source = Source.fromFile("data.txt")
  for (line <- source.getLines()) {
    array(idx) = line.toInt
    idx += 1
  }

  // convert the Scala integer array into the Chisel type Vec
  val table = VecInit(array.map(_.U(8.W)))

  // use the table
  io.data := table(io.address)
}
```

代码清单 10.1: 读取文本文件以生成逻辑表。

我们可以利用 Scala 的全部能力来生成逻辑（表）。例如，生成表示三角函数的定点常量表、计算数字滤波器的系数，或者用 Scala 编写一个小型汇编器，为用 Chisel 编写的微处理器生成代码。所有这些功能都在同一代码库（同一语言）中，可以在硬件生成期间执行。

一个经典的例子是把二进制数转换为二进制编码的十进制（BCD）表示。BCD 用 4 位表示一个十进制数字，以十进制格式表示数值。例如，十进制 13 的二进制是 1101，BCD 编码为二进制的 1 和 3：00010011。BCD 可以用十进制显示数字，比十六进制更友好。

我们可以编写一个 Java 程序来计算二进制到 BCD 的转换表，该程序输出可包含在项目中的 VHDL 代码。这个 Java 程序大约有 100 行，大部分代码用于生成 VHDL 字符串，而转换的关键部分只有两行。

使用 Chisel，可以在硬件生成过程中直接计算这张表。代码清单 10.2 展示了二进制到 BCD 转换的表生成。

```
import chisel3._

class BcdTable extends Module {
  val io = IO(new Bundle {
    val address = Input(UInt(8.W))
    val data = Output(UInt(8.W))
  })

  val array = new Array[Int](256)

  // Convert binary to BCD
  for (i <- 0 to 99) {
    array(i) = ((i/10)<<4) + i%10
  }

  val table = VecInit(array.map(_.U(8.W)))
  io.data := table(io.address)
}
```

代码清单 10.2： 二进制到二进制编码十进制（BCD）的转换。

10.3 使用继承

Chisel 是面向对象语言。硬件组件 Chisel `Module` 是一个 Scala 类。因此，我们可以使用继承把公共行为提取到父类中。下面通过一个例子探讨如何使用继承。

在第 7.2 节中，我们探讨了多种形式的计数器，可用于低频节拍生成。假设我们想比较这些不同版本的资源需求。首先定义一个抽象类，规定节拍发生器的接口：

```
abstract class Ticker(n: Int) extends Module {
  val io = IO(new Bundle{
    val tick = Output(Bool())
  })
}
```

代码清单 10.3 展示了该抽象类的第一个实现：用向上计数的计数器生成节拍。

```
class UpTicker(n: Int) extends Ticker(n) {

  val N = (n-1).U

  val cntReg = RegInit(0.U(8.W))

  cntReg := cntReg + 1.U
  when(cntReg === N) {
    cntReg := 0.U
  }

  io.tick := cntReg === N
}
```

代码清单 10.3： 用计数器生成节拍。

我们可以用同一个测试平台测试 *ticker* 逻辑的所有不同版本。我们只需让测试平台接受 `Ticker` 的子类型。代码清单 10.4 展示了测试器的 Chisel 代码。`TickerTester` 有几个参数：(1) 类型参数 `[T <: Ticker]`，用于接受 `Ticker` 或任何继承自 `Ticker` 的类；(2) 被测设计，其类型为 `T` 或其子类型；(3) 每个节拍期望的时钟周期数。测试器等待节拍第一次出现（不同实现的起始时刻可能不同），然后检查 `tick` 是否每隔 n 个时钟周期重复一次。

```
import chisel3.iotesters.PeekPokeTester
import org.scalatest._

class TickerTester[T <: Ticker](dut: T, n: Int) extends PeekPokeTester(dut:
    T) {
```

```
  // -1 is the notion that we have not yet seen the first tick
  var count = -1
  for (i <- 0 to n * 3) {
    if (count > 0) {
      expect(dut.io.tick, 0)
    }
    if (count == 0) {
      expect(dut.io.tick, 1)
    }
    val t = peek(dut.io.tick)
    // On a tick we reset the tester counter to N-1,
    // otherwise we decrement the tester counter
    if (t == 1) {
      count = n-1
    } else {
      count -= 1
    }

    step(1)
  }
}
```

代码清单 10.4: 用于测试不同版本 ticker 的测试器。

用第一个简单的 ticker 实现，我们可以先测试测试器本身，也许加上一些 `println` 调试。确信简单的 ticker 和测试器正确后，可以继续探讨另外两个 ticker 版本。代码清单 10.5 展示了用递减到 0 的计数器生成节拍；代码清单 10.6 展示了递减到 -1 的极客版本，它通过省去比较器来减少硬件开销。

```
class DownTicker(n: Int) extends Ticker(n) {

  val N = (n-1).U

  val cntReg = RegInit(N)

  cntReg := cntReg - 1.U
  when(cntReg === 0.U) {
    cntReg := N
  }

  io.tick := cntReg === N
}
```

代码清单 10.5: 用递减计数器生成节拍。

```
class NerdTicker(n: Int) extends Ticker(n) {

  val N = n

  val MAX = (N - 2).S(8.W)
  val cntReg = RegInit(MAX)
  io.tick := false.B
```

```
  cntReg := cntReg - 1.S
  when(cntReg(7)) {
    cntReg := MAX
    io.tick := true.B
  }
}
```

代码清单 10.6: 通过递减到 -1 生成节拍。

我们可以编写 ScalaTest 测试规范，创建不同版本的 ticker 实例，传给通用测试平台，从而测试全部三个版本。代码清单 10.7 展示了该测试规范。只运行 ticker 测试的命令如下：

```
sbt "testOnly TickerSpec"
```

```
class TickerSpec extends FlatSpec with Matchers {

  "UpTicker 5" should "pass" in {
    chisel3.iotesters.Driver(() => new UpTicker(5)) { c =>
      new TickerTester(c, 5)
    } should be (true)
  }

  "DownTicker 7" should "pass" in {
    chisel3.iotesters.Driver(() => new DownTicker(7)) { c =>
      new TickerTester(c, 7)
    } should be (true)
  }

  "NerdTicker 11" should "pass" in {
    chisel3.iotesters.Driver(() => new NerdTicker(11)) { c =>
      new TickerTester(c, 11)
    } should be (true)
  }
}
```

代码清单 10.7: ticker 测试的 ScalaTest 测试规范。

第十一章　设计实例

本章探讨一些小型数字设计，例如 FIFO 缓冲区，它们可以作为更大设计的构建模块。作为另一个例子，我们设计一个串口（也称为 UART），它本身可能会用到 FIFO 缓冲区。

11.1　FIFO 缓冲区

我们可以在写入方（发送方）和读取方（接收方）之间加入一个缓冲区，使二者解耦。一种常见的缓冲区是先进先出（FIFO）缓冲区。图 11.1 展示了一个写入方、一个 FIFO 和一个读取方。写入方在 `write` 信号有效时，把 `din` 上的数据写入 FIFO；读取方在 `read` 信号有效时，从 `dout` 读出数据。

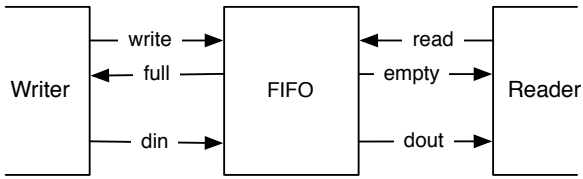


图 11.1：写入方、FIFO 缓冲区和读取方。

FIFO 初始为空，由 `empty` 信号表示。从空 FIFO 读取通常是未定义行为。如果写入数据后一直不读取，FIFO 会变为 `full`。向满 FIFO 写入通常会被忽略，数据丢失。换句话说，`empty` 和 `full` 信号充当握手信号。

FIFO 可以有多种不同实现：例如使用片上存储器加读写指针，或者只用一条寄存器链加一个小型状态机。对于小型缓冲区（最多几十个元素），把独立的寄存器连成一条缓冲链是一种简单、资源需求低的实现。气泡 FIFO 的代码可在 chisel-examples 仓库中找到。[1]

首先定义写入方和读取方的 IO 信号。数据宽度由 `size` 配置。写入数据为 `din`，写入由 `write` 表示。信号 `full` 在写入方一侧执行流控。

```
class WriterIO(size: Int) extends Bundle {
  val write = Input(Bool())
  val full = Output(Bool())
  val din = Input(UInt(size.W))
}
```

读取方通过 `dout` 提供数据，读取由 `read` 发起。`empty` 信号负责读取方一侧的流控。

```
class ReaderIO(size: Int) extends Bundle {
  val read = Input(Bool())
  val empty = Output(Bool())
```

[1]为完整起见，Chisel 本书的仓库中也包含一份 FIFO 代码副本。

```
  val dout = Output(UInt(size.W))
}
```

代码清单 11.1 展示了一个单级缓冲区。该缓冲区有一个 `WriterIO` 类型的入队端口 `enq`，以及一个 `ReaderIO` 类型的出队端口 `deq`。缓冲区的状态元件包括一个保存数据的寄存器（`dataReg`）和一个用于简单 FSM 的状态寄存器（`stateReg`）。该 FSM 只有两个状态：缓冲区空或满。如果缓冲区为空，一次写入会寄存输入数据并转到满状态；如果缓冲区为满，一次读取会取走数据并转到空状态。IO 端口 `full` 和 `empty` 分别向写入方和读取方表示缓冲区状态。

```
class FifoRegister(size: Int) extends Module {
  val io = IO(new Bundle {
    val enq = new WriterIO(size)
    val deq = new ReaderIO(size)
  })

  val empty :: full :: Nil = Enum(2)
  val stateReg = RegInit(empty)
  val dataReg = RegInit(0.U(size.W))

  when(stateReg === empty) {
    when(io.enq.write) {
      stateReg := full
      dataReg := io.enq.din
    }
  }.elsewhen(stateReg === full) {
    when(io.deq.read) {
      stateReg := empty
      dataReg := 0.U // just to better see empty slots in the waveform
    }
  }.otherwise {
    // There should not be an otherwise state
  }

  io.enq.full := (stateReg === full)
  io.deq.empty := (stateReg === empty)
  io.deq.dout := dataReg
}
```

代码清单 11.1: 气泡 FIFO 的单级缓冲区。

代码清单 11.2 展示了完整的 FIFO。完整 FIFO 的 IO 接口与单个 FIFO 缓冲区相同。`BubbleFifo` 的参数包括数据字宽 `size` 和缓冲级数 `depth`。我们可以用 `depth` 个 `FifoRegister` 构建一个 `depth` 级气泡 FIFO。这些级被放入一个 Scala `Array` 中创建。Scala 数组没有硬件含义，它只是提供了一个容器，用来保存对所创建缓冲区的引用。然后用 Scala `for` 循环连接各个缓冲区。第一个缓冲区的入队侧连接到完整 FIFO 的入队 IO，最后一个缓冲区的出队侧连接到完整 FIFO 的出队侧。

```
class BubbleFifo(size: Int, depth: Int) extends Module {
  val io = IO(new Bundle {
```

```
    val enq = new WriterIO(size)
    val deq = new ReaderIO(size)
  })

  val buffers = Array.fill(depth) { Module(new FifoRegister(size)) }
  for (i <- 0 until depth - 1) {
    buffers(i + 1).io.enq.din := buffers(i).io.deq.dout
    buffers(i + 1).io.enq.write := ~buffers(i).io.deq.empty
    buffers(i).io.deq.read := ~buffers(i + 1).io.enq.full
  }
  io.enq <> buffers(0).io.enq
  io.deq <> buffers(depth - 1).io.deq
}
```

代码清单 11.2: 由一组 FIFO 气泡级组成的 FIFO。

这种把独立缓冲区连接起来实现 FIFO 队列的思路称为气泡 FIFO，因为数据像气泡一样穿过队列。这种实现简单，在数据速率明显低于时钟频率时是一种很好的方案，例如作为下一节介绍的串口解耦缓冲区。

然而，当数据速率接近时钟频率时，气泡 FIFO 有两个限制：(1) 每个缓冲区的状态必须在空和满之间切换，这意味着 FIFO 的最大吞吐量是每个字 2 个时钟周期；(2) 数据需要穿过整个 FIFO，因此从输入到输出的延迟至少等于缓冲区级数。我将在第 11.3 节介绍其他可能的 FIFO 实现。

11.2 串口

串口（也称为 UART 或 RS-232）是在笔记本电脑和 FPGA 开发板之间通信的最简单方式之一。顾名思义，数据以串行方式传输。一个 8 位字节按如下格式传输：一个起始位（0）、8 位数据（最低有效位在前），然后是一到两个停止位（1）。没有数据传输时，输出为 1。图 11.2 展示了一个字节传输的时序图。

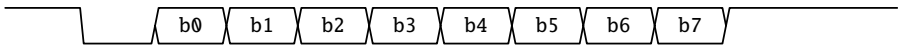


图 11.2: UART 传输一个字节。

我们以模块化方式设计 UART，每个模块的功能尽量少。先介绍发送器（TX）、接收器（RX）、缓冲区，然后介绍这些基础组件的使用。

首先需要一个接口，即端口定义。在 UART 设计中，我们使用 ready/valid 握手接口，方向以发送器的视角给出。

```
class Channel extends Bundle {
  val data = Input(Bits(8.W))
  val ready = Output(Bool())
```

```
  val valid = Input(Bool())
}
```

ready/valid 接口的约定是：当 `ready` 和 `valid` 都置位时，数据传输发生。

```
class Tx(frequency: Int, baudRate: Int) extends Module {
  val io = IO(new Bundle {
    val txd = Output(Bits(1.W))
    val channel = new Channel()
  })

  val BIT_CNT = ((frequency + baudRate / 2) / baudRate - 1).asUInt()

  val shiftReg = RegInit(0x7ff.U)
  val cntReg = RegInit(0.U(20.W))
  val bitsReg = RegInit(0.U(4.W))

  io.channel.ready := (cntReg === 0.U) && (bitsReg === 0.U)
  io.txd := shiftReg(0)

  when(cntReg === 0.U) {

    cntReg := BIT_CNT
    when(bitsReg =/= 0.U) {
      val shift = shiftReg >> 1
      shiftReg := Cat(1.U, shift(9, 0))
      bitsReg := bitsReg - 1.U
    }.otherwise {
      when(io.channel.valid) {
        // two stop bits, data, one start bit
        shiftReg := Cat(Cat(3.U, io.channel.data), 0.U)
        bitsReg := 11.U
      }.otherwise {
        shiftReg := 0x7ff.U
      }
    }

  }.otherwise {
    cntReg := cntReg - 1.U
  }
}
```

代码清单 11.3：串口发送器。

代码清单 11.3 展示了一个最简的串行发送器（`Tx`）。IO 端口包括发送串行数据的 `txd`，以及一个 `Channel`，发送器通过它接收要串行化并发送的字符。为了产生正确的时序，我们计算一个常量：一个串行位对应的时钟周期数。

我们使用三个寄存器：（1）用于移位（串行化）数据的寄存器（`shiftReg`）；（2）用于产生正确波特率的计数器（`cntReg`）；（3）用于记录还需移出多少位的计数器。不需要额外的 FSM 状态寄存器，所有状态都编码在这三个寄存器中。

计数器 `cntReg` 连续运行（递减到 0，到 0 时复位到起始值）。所有动作只在 `cntReg` 为 0

时执行。由于我们构建的是最简发送器，只有移位寄存器用于保存数据。因此，只有当 cntReg 为 0 且没有剩余位需要移出时，通道才处于 ready 状态。

IO 端口 txd 直接连接到移位寄存器的最低有效位。

当还有位需要移出（bitsReg =/= 0.U）时，我们右移一位，并从高位填入 1（发送器的空闲电平）。如果没有更多位需要移出，就检查通道中是否有数据（由 valid 端口表示）。如果有，则构造要移出的位串：一个起始位（0）、8 位数据和两个停止位（1）。因此，位计数器设为 11。

这个最简发送器没有额外的缓冲区，只有当移位寄存器为空、且 cntReg 为 0 的那个时钟周期才能接收新字符。只在 cntReg 为 0 时接收新数据，也意味着即使移位寄存器中还有空间，ready 标志也会被撤销。不过，我们不想给发送器增加这种“复杂度”，而是把它交给缓冲区处理。

```
class Buffer extends Module {
  val io = IO(new Bundle {
    val in = new Channel()
    val out = Flipped(new Channel())
  })

  val empty :: full :: Nil = Enum(2)
  val stateReg = RegInit(empty)
  val dataReg = RegInit(0.U(8.W))

  io.in.ready := stateReg === empty
  io.out.valid := stateReg === full

  when(stateReg === empty) {
    when(io.in.valid) {
      dataReg := io.in.data
      stateReg := full
    }
  }.otherwise { // full
    when(io.out.ready) {
      stateReg := empty
    }
  }
  io.out.data := dataReg
}
```

代码清单 11.4： 带 ready/valid 接口的单字节缓冲区。

代码清单 11.4 展示了一个单字节缓冲区，类似于气泡 FIFO 中的 FIFO 寄存器。输入端口是 Channel 接口，输出是方向翻转的 Channel 接口。缓冲区包含一个最小的状态机，用来表示空或满。缓冲区驱动的握手信号（in.ready 和 out.valid）取决于状态寄存器。

当状态为空且输入数据 valid 时，寄存数据并转到满状态。当状态为满且下游接收方 ready 时，下游数据传输发生，并转回空状态。

```
class BufferedTx(frequency: Int, baudRate: Int) extends Module {
  val io = IO(new Bundle {
```

```
    val txd = Output(Bits(1.W))
    val channel = new Channel()
  })
  val tx = Module(new Tx(frequency, baudRate))
  val buf = Module(new Buffer())

  buf.io.in <> io.channel
  tx.io.channel <> buf.io.out
  io.txd <> tx.io.txd
}
```

代码清单 11.5： 带额外缓冲区的发送器。

有了这个缓冲区，就可以扩展我们的最简发送器。代码清单 11.5 展示了发送器 `Tx` 前面加一个单字节缓冲区的组合。这个缓冲区缓解了 `Tx` 只在单个时钟周期内 ready 的问题。我们把该问题的解决方案交给了缓冲区模块。把单字缓冲区扩展为真正的 FIFO 很容易，发送器和单字节缓冲区都无需改动。

```
class Rx(frequency: Int, baudRate: Int) extends Module {
  val io = IO(new Bundle {
    val rxd = Input(Bits(1.W))
    val channel = Flipped(new Channel())
  })

  val BIT_CNT = ((frequency + baudRate / 2) / baudRate - 1).U
  val START_CNT = ((3 * frequency / 2 + baudRate / 2) / baudRate - 1).U

  // Sync in the asynchronous RX data
  val rxReg = RegNext(RegNext(io.rxd))

  val shiftReg = RegInit('A'.U(8.W))
  val cntReg = RegInit(0.U(20.W))
  val bitsReg = RegInit(0.U(4.W))
  val valReg = RegInit(false.B)

  when(cntReg =/= 0.U) {
    cntReg := cntReg - 1.U
  }.elsewhen(bitsReg =/= 0.U) {
    cntReg := BIT_CNT
    shiftReg := Cat(rxReg, shiftReg >> 1)
    bitsReg := bitsReg - 1.U
    // the last shifted in
    when(bitsReg === 1.U) {
      valReg := true.B
    }
  // wait 1.5 bits after falling edge of start
  }.elsewhen(rxReg === 0.U) {
    cntReg := START_CNT
    bitsReg := 8.U
  }

  when(valReg && io.channel.ready) {
    valReg := false.B
```

```
  }

  io.channel.data := shiftReg
  io.channel.valid := valReg
}
```

代码清单 11.6: 串口接收器。

代码清单 11.6 展示了接收器（Rx）的代码。接收器稍微棘手一些，因为它需要重建串行数据的时序。接收器等待起始位的下降沿。从该事件起，接收器等待 1.5 个位时间，使自己位于第 0 位的中间。然后每隔一个位时间移入一位。可以看到这两个等待时间分别由 START_CNT 和 BIT_CNT 表示。两个时间共用同一个计数器（cntReg）。移入 8 位后，valReg 表示有一个字节可用。

```
class Sender(frequency: Int, baudRate: Int) extends Module {
  val io = IO(new Bundle {
    val txd = Output(Bits(1.W))
  })

  val tx = Module(new BufferedTx(frequency, baudRate))

  io.txd := tx.io.txd

  val msg = "Hello World!"
  val text = VecInit(msg.map(_.U))
  val len = msg.length.U

  val cntReg = RegInit(0.U(8.W))

  tx.io.channel.data := text(cntReg)
  tx.io.channel.valid := cntReg =/= len

  when(tx.io.channel.ready && cntReg =/= len) {
    cntReg := cntReg + 1.U
  }
}
```

代码清单 11.7: 通过串口发送“Hello World!”。

代码清单 11.7 展示了如何使用串口发送器发出一条友好的消息。我们在 Scala 字符串（msg）中定义消息，并把它转换为 Chisel UInt 组成的 Vec。Scala 字符串是一个支持 map 方法的序列。map 方法接收一个函数字面量，对每个元素应用该函数，并构建由函数返回值组成的序列。如果函数字面量只有一个参数（本例就是如此），可以用 _ 表示该参数。我们的函数字面量调用 Chisel 方法.U，把 Scala Char 转换为 Chisel UInt。然后把序列传给 VecInit 构造 Chisel Vec。我们用计数器 cntReg 索引向量 text，把各个字符提供给带缓冲的发送器。每收到一个 ready 信号，计数器递增，直到整个字符串发送完毕。发送器一直保持 valid，直到最后一个字符发送完成。

```
class Echo(frequency: Int, baudRate: Int) extends Module {
```

```
  val io = IO(new Bundle {
    val txd = Output(Bits(1.W))
    val rxd = Input(Bits(1.W))
  })

  val tx = Module(new BufferedTx(frequency, baudRate))
  val rx = Module(new Rx(frequency, baudRate))
  io.txd := tx.io.txd
  rx.io.rxd := io.rxd
  tx.io.channel <> rx.io.channel
}
```

代码清单 11.8: 在串口上回显数据。

代码清单 11.8 展示了如何把接收器和发送器连接起来使用。这种连接构成一个 `Echo` 电路，每个接收到的字符都会被发回（回显）。

11.3 FIFO 设计的变体

本节将实现 FIFO 队列的不同变体。为了让这些实现可以互换，我们使用第 10.3 节介绍的继承。定义一个**抽象** FIFO 类：

```
abstract class Fifo[T <: Data](gen: T) extends Module {
  val io = new Bundle {
    val enq = Flipped(new DecoupledIO(gen))
    val deq = new DecoupledIO(gen)
  }
}
```

注意，我们使用 Chisel 类型作为参数，以便缓冲任意 Chisel 数据类型。在第 11.1 节中，我们为接口定义了自己的类型，信号名如 `write`、`full`、`din`、`read`、`empty` 和 `dout`。这种缓冲区的输入和输出由数据以及两个握手信号组成（例如，当 FIFO 未满时向其写入）。不过，我们可以把这种握手推广为所谓的 ready-valid 接口。例如，当 FIFO `ready` 时，可以入队一个元素（写入 FIFO）。写入方用 `valid` 表示这一点。由于 ready-valid 接口非常常见，Chisel 在 `DecoupledIO` 中给出了该接口的定义：[2]

```
  class DecoupledIO[T <: Data](gen: T) {
    val ready = Input(Bool())
    val valid = Output(Bool())
    val bits  = Output(gen)
  }
```

`DecoupledIO` 接口是从写入方（生产者）的视角定义的。因此，FIFO 的入队端口需要翻转信号方向。

[2]这里做了简化，`DecoupledIO` 实际上继承自一个抽象类。

11.4 练习

本练习部分稍长，包含两个练习：(1) 探索气泡 FIFO，并实现另一种 FIFO 设计；(2) 探索 UART 并扩展它。两个练习的源代码都包含在 chisel-examples 仓库中。

11.4.1 探索 FIFO 变体

FIFO 源码还包含一个测试器，它会触发不同的读写行为，并生成值变化转储（VCD）格式的波形。VCD 文件可以用波形查看器查看，例如 GTKWave。请浏览仓库中的 FifoTester。仓库包含用于运行示例的 `Makefile`，对于 FIFO 示例只需输入：

```
$ make fifo
```

这条 make 命令会编译 FIFO、运行测试，并启动 GTKWave 查看波形。请浏览测试器和生成的波形。

在最初的几个周期，测试器写入一个字。可以观察这个字如何在 FIFO 中像气泡一样穿过，这也是气泡 *FIFO* 名称的由来。这种“冒泡”也意味着数据字穿过 FIFO 的延迟等于 FIFO 的深度。

下一个测试把 FIFO 填满，然后执行一次读取。注意“空”状态如何从 FIFO 的读取端冒泡到写入端。当气泡 FIFO 满时，一次读取需要经过与缓冲深度相当的延迟才能影响写入端。

测试末尾包含一个循环，尝试以最高速度读写。可以看到气泡 FIFO 以最大带宽运行，即每个字两个时钟周期。每次单字传输时，缓冲级都必须在空和满之间切换。

气泡 FIFO 简单，且对于小型缓冲区资源需求低。n 级气泡 FIFO 的主要缺点是：(1) 最大吞吐量为每两个时钟周期一个字；(2) 数据字需要 n 个时钟周期才能从写入端传到读取端；(3) 满 FIFO 需要 n 个时钟周期才能重新开始。

这些缺点可以用环形缓冲区实现的 FIFO 来解决。环形缓冲区可以用一个存储器以及读写指针实现。请用环形缓冲区实现一个包含四个元素、接口相同的 FIFO，并用测试器探索其不同行为。作为环形缓冲区的初始实现，可以走捷径，使用一个寄存器向量（`Reg(Vec(4, UInt(size.W)))`）。

11.4.2 UART

对于 UART 示例，需要一块带串口的 FPGA 开发板和笔记本电脑的串口（通常通过 USB 连接）。把串口线连接在 FPGA 开发板和笔记本电脑的串口之间。启动一个终端程序，例如 Windows 上的 Hyperterm 或 Linux 上的 `gtkterm`：

```
$ gtkterm &
```

配置端口以使用正确的设备，对于 USB 转串口，通常是 `/dev/ttyUSB0` 之类。把波特率设为 115200，无校验、无流控（握手）。用以下命令生成 UART 的 Verilog 代码：

```
$ make uart
```

然后用综合工具综合该设计。仓库中包含面向 DE2-115 FPGA 开发板的 Quartus 工程。在 Quartus 中点击 play 按钮综合设计，然后配置 FPGA。配置完成后，终端中应该会显示一条问候消息。

为闪烁 LED 示例增加 UART，在 LED 熄灭和点亮时分别向串口写入 0 和 1。像 `Sender` 示例那样使用 `BufferedTx`。

字符输出很慢（每秒两个），因此可以直接把数据写入 UART 发送寄存器，忽略 ready/valid 握手。扩展该示例，以波特率允许的最快速度重复写入数字 0-9。这种情况下，需要扩展状态机来轮询 UART 状态，检查发送缓冲区是否空闲。

示例代码中 `Tx` 只有一个缓冲区。你可以把自己实现的 FIFO 加进来，为发送器和接收器增加缓冲。

第十二章　处理器设计

作为本书最后几章之一，我们介绍一个中等规模的项目：微处理器的设计、仿真与测试。为了让项目规模可控，我们设计一台简单的累加器机器。该处理器名为 Leros [8]，其开源代码位于 https://github.com/leros-dev/leros。需要说明的是，这是一个进阶示例，理解其中的代码示例需要一定的计算机体系结构知识。

Leros 的设计目标是简单，同时仍能作为 C 编译器的合适目标。其指令描述可以放在一页内，见表 12.1。表中 `A` 表示累加器，`PC` 是程序计数器，`i` 是立即数（0 到 255），`Rn` 是寄存器 `n`（0 到 255），`o` 是相对于 `PC` 的跳转偏移，`AR` 是用于访存的地址寄存器。

12.1　从 ALU 开始

处理器的核心组件之一是算术逻辑单元，简称 ALU。因此，我们从 ALU 的编码和测试平台开始。首先定义一个 `Enum` 来表示 ALU 的不同操作：

```
object Types {
  val nop :: add :: sub :: and :: or :: xor :: ld :: shr :: Nil = Enum(8)
}
```

ALU 通常有两个操作数输入（称为 `a` 和 `b`）、一个用于选择功能的操作码 `op` 输入，以及一个输出 `y`。代码清单 12.1 展示了该 ALU。

我们首先为三个输入定义较短的名称。`switch` 语句定义 `res` 的计算逻辑，因此先给它一个默认赋值 0。switch 语句枚举所有操作，并相应地赋值表达式。所有操作都直接映射为 Chisel 表达式。最后，把结果 `res` 赋给 ALU 输出 `y`。

```
class Alu(size: Int) extends Module {
  val io = IO(new Bundle {
    val op = Input(UInt(3.W))
    val a = Input(SInt(size.W))
    val b = Input(SInt(size.W))
    val y = Output(SInt(size.W))
  })

  val op = io.op
  val a = io.a
  val b = io.b
  val res = WireInit(0.S(size.W))

  switch(op) {
    is(add) {
      res := a + b
    }
    is(sub) {
      res := a - b
    }
    is(and) {
      res := a & b
```

操作码	功能	说明
add	A = A + Rn	将寄存器 Rn 加到 A
addi	A = A + i	将立即数 i 加到 A
sub	A = A - Rn	从 A 中减去寄存器 Rn
subi	A = A - i	从 A 中减去立即数 i
shr	A = A >>> 1	将 A 逻辑右移一位
load	A = Rn	将寄存器 Rn 装入 A
loadi	A = i	将立即数 i 装入 A
and	A = A and Rn	将寄存器 Rn 与 A 相与
andi	A = A and i	将立即数 i 与 A 相与
or	A = A or Rn	将寄存器 Rn 与 A 相或
ori	A = A or i	将立即数 i 与 A 相或
xor	A = A xor Rn	将寄存器 Rn 与 A 相异或
xori	A = A xor i	将立即数 i 与 A 相异或
loadhi	A_{15-8} = i	将立即数装入第二个字节
loadh2i	A_{23-16} = i	将立即数装入第三个字节
loadh3i	A_{31-24} = i	将立即数装入第四个字节
store	Rn = A	将 A 存入寄存器 Rn
jal	PC = A, Rn = PC + 2	跳转到 A，并将返回地址存入 Rn
ldaddr	AR = A	将 A 装入地址寄存器 AR
loadind	A = mem[AR+(i << 2)]	从存储器读取一个字装入 A
loadindbu	A = mem[AR+i]$_{7-0}$	从存储器读取一个无符号字节装入 A
storeind	mem[AR+(i << 2)] = A	将 A 存入存储器
storeindb	mem[AR+i]$_{7-0}$ = A	将一个字节存入存储器
br	PC = PC + o	跳转
brz	if A == 0 PC = PC + o	若 A 为零则跳转
brnz	if A != 0 PC = PC + o	若 A 非零则跳转
brp	if A $\geq$ 0 PC = PC + o	若 A 为正则跳转
brn	if A $<$ 0 PC = PC + o	若 A 为负则跳转
scall	scall A	系统调用（仿真钩子）

表 12.1： Leros 指令集。

```
}
is(or) {
  res := a | b
}
is(xor) {
  res := a ^ b
}
is (shr) {
  // the following does NOT result in an unsigned shift
```

```
      // res := (a.asUInt >> 1).asSInt
      // work around
      res := (a >> 1) & 0x7fffffff.S
    }
    is(ld) {
      res := b
    }
  }

  io.y := res
}
```

代码清单 12.1: Leros 的 ALU。

为了测试，我们用纯 Scala 编写 ALU 的功能，如代码清单 12.2 所示。

```
  def alu(a: Int, b: Int, op: Int): Int = {

    op match {
      case 1 => a + b
      case 2 => a - b
      case 3 => a & b
      case 4 => a | b
      case 5 => a ^ b
      case 6 => b
      case 7 => a >>> 1
      case _ => -123 // This shall not happen
    }
  }
```

代码清单 12.2: 用 Scala 编写的 Leros ALU 功能。

这种用 Scala 实现来对照 Chisel 硬件描述的方式，虽然无法发现规格说明本身的错误，但至少可以作为一种合理性检查。我们使用一些边界情况的值作为测试向量：

```
  // Some interesting corner cases
  val interesting = Array(1, 2, 4, 123, 0, -1, -2, 0x80000000, 0x7fffffff)
```

我们用这些值作为两个输入，测试所有功能：

```
  def test(values: Seq[Int]) = {
    for (fun <- add to shr) {
      for (a <- values) {
        for (b <- values) {
          poke(dut.io.op, fun)
          poke(dut.io.a, a)
          poke(dut.io.b, b)
          step(1)
          expect(dut.io.y, alu(a, b, fun.toInt))
        }
      }
    }
  }
```

对 32 位参数进行完全穷举测试是不可能的，这也是我们选择一些边界情况作为输入值的原因。除了边界情况，用随机输入进行测试也很有用：

```
  val randArgs = Seq.fill(100)(scala.util.Random.nextInt)
  test(randArgs)
```

可以在 Leros 项目中用以下命令运行测试：

```
$ sbt "test:runMain leros.AluTester"
```

应该会输出类似下面的成功消息：

```
[info] [0.001] SEED 1544507337402
test Alu Success: 70567 tests passed in 70572 cycles taking
3.845715 seconds
[info] [3.825] RAN 70567 CYCLES PASSED
```

12.2 指令译码

从 ALU 出发，我们反向工作，实现指令译码器。不过，首先在独立的 Scala 类和共享包中定义指令编码。我们希望在 Leros 的硬件实现、Leros 汇编器和 Leros 指令集模拟器之间共享编码常量。

```
package leros.shared {

object Constants {
  val NOP = 0x00
  val ADD = 0x08
  val ADDI = 0x09
  val SUB = 0x0c
  val SUBI = 0x0d
  val SHR = 0x10
  val LD = 0x20
  val LDI = 0x21
  val AND = 0x22
  val ANDI = 0x23
  val OR = 0x24
  val ORI = 0x25
  val XOR = 0x26
  val XORI = 0x27
  val LDHI = 0x29
  val LDH2I = 0x2a
  val LDH3I = 0x2b
  val ST = 0x30
  // ...
```

对于译码组件，我们定义一个用于输出的 `Bundle`，后续会将其中的一部分送入 ALU。

```
class DecodeOut extends Bundle {
  val ena = Bool()
  val func = UInt()
  val exit = Bool()
}
```

译码器接收 8 位操作码作为输入，并输出译码后的信号。这些控制信号先用 `WireInit` 赋予默认值。

```
class Decode() extends Module {
  val io = IO(new Bundle {
    val din = Input(UInt(8.W))
    val dout = Output(new DecodeOut)
  })

  val f = WireInit(nop)
  val imm = WireInit(false.B)
  val ena = WireInit(false.B)

  io.dout.exit := false.B
```

译码本身就是一个大型 switch 语句，作用于指令中表示操作码的部分（在 Leros 中，大多数指令的操作码位于高 8 位）。

```
  switch(io.din) {
    is(ADD.U) {
      f := add
      ena := true.B
    }
    is(ADDI.U) {
      f := add
      imm := true.B
      ena := true.B
    }
    is(SUB.U) {
      f := sub
      ena := true.B
    }
    is(SUBI.U) {
      f := sub
      imm := true.B
      ena := true.B
    }
    is(SHR.U) {
      f := shr
      ena := true.B
    }
    // ...
```

12.3 汇编指令

要为 Leros 编写程序，需要一个汇编器。不过，在最开始的测试中，可以硬编码几条指令，放入一个 Scala 数组，用来初始化指令存储器。

```
val prog = Array[Int](
  0x0903, // addi 0x3
  0x09ff, // -1
  0x0d02, // subi 2
  0x21ab, // ldi 0xab
  0x230f, // and 0x0f
  0x25c3, // or 0xc3
  0x0000
)

def getProgramFix() = prog
```

不过，这种测试处理器的方法效率很低。用 Scala 这样富有表现力的语言编写汇编器并不是一个大项目。因此，我们为 Leros 编写一个简单的汇编器，大约 100 行代码即可完成。我们定义一个调用汇编器的函数 `getProgram`。对于跳转目标，需要一个符号表，用 `Map` 收集。经典汇编器分两遍运行：（1）收集符号表中的值；（2）使用第一遍收集的符号汇编程序。因此，我们调用 `assemble` 两次，并用参数指明当前是第几遍。

```
def getProgram(prog: String) = {
  assemble(prog)
}

// collect destination addresses in first pass
val symbols = collection.mutable.Map[String, Int]()

def assemble(prog: String): Array[Int] = {
  assemble(prog, false)
  assemble(prog, true)
}
```

`assemble` 函数首先读入源文件[1]，然后定义两个辅助函数来解析两种可能的操作数：（1）整数常量（允许十进制或十六进制表示）；（2）读取寄存器编号。

```
def assemble(prog: String, pass2: Boolean): Array[Int] = {

  val source = Source.fromFile(prog)
  var program = List[Int]()
  var pc = 0

  def toInt(s: String): Int = {
    if (s.startsWith("0x")) {
      Integer.parseInt(s.substring(2), 16)
    } else {
```

[1]这个函数实际上并不读取源文件，但在本讨论中我们可以把它视为读取函数。

```
      Integer.parseInt(s)
    }
  }

  def regNumber(s: String): Int = {
    assert(s.startsWith("r"), "Register numbers shall start with \'r\'")
    s.substring(1).toInt
  }
```

代码清单 12.3 展示了 Leros 汇编器的核心。Scala 的 `match` 表达式覆盖了汇编函数的核心部分。

```
  for (line <- source.getLines()) {
    if (!pass2) println(line)
    val tokens = line.trim.split(" ")
    val Pattern = "(.*:)".r
    val instr = tokens(0) match {
      case "//" => // comment
      case Pattern(l) => if (!pass2) symbols += (l.substring(0, l.length -
          1) -> pc)
      case "add" => (ADD << 8) + regNumber(tokens(1))
      case "sub" => (SUB << 8) + regNumber(tokens(1))
      case "and" => (AND << 8) + regNumber(tokens(1))
      case "or" => (OR << 8) + regNumber(tokens(1))
      case "xor" => (XOR << 8) + regNumber(tokens(1))
      case "load" => (LD << 8) + regNumber(tokens(1))
      case "addi" => (ADDI << 8) + toInt(tokens(1))
      case "subi" => (SUBI << 8) + toInt(tokens(1))
      case "andi" => (ANDI << 8) + toInt(tokens(1))
      case "ori" => (ORI << 8) + toInt(tokens(1))
      case "xori" => (XORI << 8) + toInt(tokens(1))
      case "shr" => (SHR << 8)
      // ...
      case "" => // println("Empty line")
      case t: String => throw new Exception("Assembler error: unknown
          instruction: " + t)
      case _ => throw new Exception("Assembler error")
```

代码清单 12.3: Leros 汇编器的主要部分。

12.4 练习

作为最后几章之一的练习，本次作业形式非常自由。你已经完成了 Chisel 的学习之旅，准备好处理自己感兴趣的设计问题了。

一个选择是重读本章，并对照 Leros 仓库中的全部源代码阅读，运行测试用例，故意改坏代码，观察测试如何失败。

另一个选择是编写自己的 Leros 实现。仓库中的实现只是流水线的一种可能组织方式。你可以编写只有单级流水线的 Chisel 仿真版本，也可以大胆地把 Leros 超级流水化，以获得尽可

能高的时钟频率。

第三个选择是从零开始设计自己的处理器。也许 Leros 处理器及其所需工具的演示已经让你相信：处理器设计与实现并非魔法，而是一项可以带来很多乐趣的工程工作。

第十三章　为 Chisel 做贡献

Chisel 是一个持续开发和改进的开源项目，因此你也可以为项目做贡献。本章介绍如何搭建 Chisel 库开发环境，以及如何为 Chisel 做贡献。

13.1　搭建开发环境

Chisel 由多个不同的仓库组成，全部托管在 GitHub 上的 freechips 组织下。

把你想要贡献的仓库 fork 到自己的 GitHub 账户。可以在 GitHub 网页界面中点击 `Fork` 按钮来 fork 仓库。然后从该 fork 克隆你的仓库副本。在我们的例子中，我们修改 `chisel3`，克隆我本地 fork 的命令是：

```
$ git clone git@github.com:schoeberl/chisel3.git
```

要编译 Chisel 3 并发布为本地库，请执行：

```
$ cd chisel3
$ sbt compile
$ sbt publishLocal
```

在执行 publishLocal 时，注意发布库的版本字符串，其中包含 `SNAPSHOT`。如果使用测试器时，发布的版本与 Chisel SNAPSHOT 不兼容，请同样 fork 并克隆 chisel-tester 仓库，并将其发布到本地。

要在 Chisel 中测试你的修改，可能还需要搭建一个 Chisel 项目，例如 fork/克隆一个空的 Chisel 项目，重命名，并删除其中的 `.git` 文件夹。

修改 `build.sbt`，引用本地发布的 Chisel 版本。此外，在本书写作时，Chisel 源码主干使用 Scala 2.12，但 Scala 2.12 在处理匿名 Bundle 时存在问题。因此需要添加以下 Scala 选项：`"-Xsource:2.11"`。`build.sbt` 应类似这样：

```
scalaVersion := "2.12.6"

scalacOptions := Seq("-Xsource:2.11")

resolvers ++= Seq(
  Resolver.sonatypeRepo("snapshots"),
  Resolver.sonatypeRepo("releases")
)

libraryDependencies +=
```

```
  "edu.berkeley.cs" %% "chisel3" % "3.2-SNAPSHOT"
libraryDependencies +=
  "edu.berkeley.cs" %% "chisel-iotesters" % "1.3-SNAPSHOT"
```

编译你的 Chisel 测试应用，并仔细确认它是否使用了本地发布的 Chisel 库版本（服务器上也发布了 SNAPSHOT 版本，因此，如果 Chisel 库和应用代码使用的 Scala 版本不同，它可能会从服务器获取 SNAPSHOT 版本，而不是你的本地发布库）。

另请参阅 Chisel 仓库中的一些说明。

13.2 测试

修改 Chisel 库后，应运行 Chisel 的测试。在基于 `sbt` 的项目中，通常运行：

```
$ sbt test
```

此外，如果为 Chisel 添加了新功能，还应为新功能提供测试。

13.3 通过 Pull Request 贡献代码

在 Chisel 项目中，开发者不会直接向主仓库提交代码。贡献通过 fork 仓库中某个分支发起的 pull request 来组织。更多信息请参阅 GitHub 上关于通过 pull request 协作的文档。Chisel 团队已经开始记录贡献指南。

13.4 练习

为 `UInt` 类型发明一个新运算符，在 Chisel 库中实现它，并编写一些使用/测试代码来探索该运算符。它不必是一个有用的运算符，任何东西都可以。例如，一个? 运算符：如果左侧不等于 0，就返回左侧，否则返回右侧。听起来像多路选择器，对吧? 你需要添加多少行代码? [1]

尽管这很简单，请不要因此动心去 fork Chisel 项目并添加你自己的小扩展。变更和扩展应与主要开发者协调。这个练习只是让你入门的一个简单练习。

如果你胆子更大了，可以挑选一个未解决的 issue 并尝试解决它，然后通过 pull request 为 Chisel 做贡献。不过，最好先观察 GitHub 仓库，了解 Chisel 的开发风格，看看 Chisel 开源项目如何处理变更和 pull request。

[1] 一个快速粗糙的实现只需要两行 Scala 代码。

第十四章　总结

本书介绍了如何使用硬件构建语言 Chisel 进行数字设计。我们看到了若干用 Chisel 描述的简单到中等规模的数字电路。Chisel 嵌入在 Scala 中，因而继承了 Scala 强大的抽象能力。由于本书是一本入门书，我们把示例限制在 Scala 的简单用法上。接下来合乎逻辑的一步是学习一些 Scala 基础知识，并将它们应用到你的 Chisel 项目中。

我很乐意收到关于本书的反馈，因为我会继续改进本书并出版新版本。你可以通过 masca@dtu.dk 联系我，或在 GitHub 仓库上提交 issue。我也很乐意接受本书仓库的 pull request，用于修正错误和改进内容。

源码获取

本书以开源形式提供。仓库还包含 Chisel 课程的幻灯片和所有 Chisel 示例：`https://github.com/schoeberl/chisel-book`

一个中等规模示例的集合（其中大部分在本书中被引用）同样开源提供。该集合还包含面向多种常见 FPGA 开发板的工程：`https://github.com/schoeberl/chisel-examples`

附录 A　Chisel 项目

Chisel（目前）还没有被许多项目使用，因此可用于学习该语言和编码风格的开源 Chisel 代码并不多。下面列出我们所知道的若干使用 Chisel 的开源项目。

Rocket Chip 是一个 RISC-V [13] 处理器复合体生成器，包含 Rocket 微体系结构和 TileLink 互连生成器。它最初在 UC Berkeley 开发，是第一个芯片级 Chisel 项目 [1]，如今由 SiFive 提供商业支持。

Sodor 是一组用于教学的 RISC-V 实现，包含 1、2、3 和 5 级流水线实现。所有处理器都使用一个简单的暂存存储器，供指令取指、数据访问和通过调试端口加载程序共享。Sodor 主要用于仿真。

Patmos 是一个面向实时系统优化的处理器实现 [10]。Patmos 仓库包含多种多核通信架构，例如时间可预测的存储器仲裁器 [7]、片上网络 [9]，以及带所有权的共享暂存存储器 [11]。在本书写作时，Patmos 仍用 Chisel 2 描述。

FlexPRET 是一个精度定时架构的实现 [14]。FlexPRET 实现 RISC-V 指令集，并已更新到 Chisel 3.1。

Lipsi 是一个小型处理器，用于实现片上系统中的实用功能 [6]。由于 Lipsi 的代码库非常小，它可以作为用 Chisel 进行处理器设计的简易起点。Lipsi 还展示了 Chisel/Scala 的生产力：作者用 14 个小时用 Chisel 描述硬件并在 FPGA 上运行，用 Scala 编写汇编器，用 Scala 编写 Lipsi 指令集模拟器用于协同仿真，还用 Lipsi 汇编器编写了一些测试用例。

OpenSoC Fabric 是一个用 Chisel 编写的开源片上网络（NoC）生成器 [5]，旨在为大规模设计空间探索提供片上系统。该 NoC 本身是一个先进的设计，采用虫洞路由、信用流控和虚拟通道。OpenSoC Fabric 仍在使用 Chisel 2。

DANA 是一个神经网络加速器，通过 Rocket 自定义协处理器（RoCC）接口与 RISC-V Rocket 处理器集成 [4]。DANA 支持推理和学习。

如果你知道使用 Chisel 的开源项目，请告诉我，以便我将其纳入本书的未来版本。

附录 B　Chisel 2

本书涵盖 Chisel 第 3 版。此外，新设计推荐使用 Chisel 3。不过，仍有一些 Chisel 2 代码尚未转换为 Chisel 3。以下文档介绍了如何把 Chisel 2 项目转换为 Chisel 3：

- Chisel2 vs. Chisel3，以及
- Towards Chisel 3

不过，你可能会参与仍在使用 Chisel 2 的项目，例如 Patmos [10] 处理器。因此，这里为从 Chisel 3 入门的读者提供一些 Chisel 2 编码信息。

首先，Chisel 官方网站上所有关于 Chisel 2 的文档都已被移除。我们抢救了那些 PDF 文档，并放在 GitHub 上：`https://github.com/schoeberl/chisel2-doc`。可以切换到 Chisel 2 分支来使用 Chisel 2 教程：

```
$ git clone https://github.com/ucb-bar/chisel-tutorial.git
$ cd chisel-tutorial
$ git checkout chisel2
```

Chisel 3 和 2 最明显的区别在于常量、IO 的 Bundle、Wire、存储器，以及可能更老形式的寄存器定义的写法。

在一定程度上，可以在 Chisel 3 项目中使用 Chisel 2 结构：使用包 `Chisel` 而不是 `chisel3` 的兼容层。不过，兼容层只应在过渡阶段使用，因此这里不再展开。

下面是两个基本组件的例子，与 Chisel 3 中介绍的相同。一个包含组合逻辑的模块：

```
import Chisel._

class Logic extends Module {
  val io = new Bundle {
    val a = UInt(INPUT, 1)
    val b = UInt(INPUT, 1)
    val c = UInt(INPUT, 1)
    val out = UInt(OUTPUT, 1)
  }

  io.out := io.a & io.b | io.c
}
```

注意，IO 定义中的 `Bundle` 没有包装进 `IO()` 类。此外，不同 IO 端口的方向作为类型定义的一部分给出，在本例中作为 `UInt` 的 `INPUT` 和 `OUTPUT` 给出。位宽作为第二个参数给出。

Chisel 2 中的 8 位寄存器示例:

```
import Chisel._

class Register extends Module {
  val io = new Bundle {
    val in = UInt(INPUT, 8)
    val out = UInt(OUTPUT, 8)
  }

  val reg = Reg(init = UInt(0, 8))
  reg := io.in

  io.out := reg
}
```

这里可以看到典型的寄存器定义：把复位值作为 `UInt` 传给命名参数 `init`。这种形式在 Chisel 3 中仍然有效，但新 Chisel 3 设计推荐使用 `RegInit` 和 `RegNext`。还要注意这里把 8 位宽的 0 定义为 `UInt(0, 8)`。

基于 Chisel 的测试会通过调用 `chiselMainTest` 和 `chiselMain` 生成 C++ 代码和 Verilog 代码。两个“main”函数都接收一个 `String` 数组作为进一步参数。

```
import Chisel._

class LogicTester(c: Logic) extends Tester(c) {

  poke(c.io.a, 1)
  poke(c.io.b, 0)
  poke(c.io.c, 1)
  step(1)
  expect(c.io.out, 1)
}

object LogicTester {
  def main(args: Array[String]): Unit = {
    chiselMainTest(Array("--genHarness", "--test",
      "--backend", "c",
      "--compile", "--targetDir", "generated"),
      () => Module(new Logic())) {
        c => new LogicTester(c)
      }
  }
}


import Chisel._

object LogicHardware {
  def main(args: Array[String]): Unit = {
    chiselMain(Array("--backend", "v"), () => Module(new Logic()))
  }
}
```

在 Chisel 2 中，具有同步寄存读写端口的存储器定义如下：

```
val mem = Mem(UInt(width = 8), 256, seqRead = true)
val rdData = mem(Reg(next = rdAddr))
when(wrEna) {
  mem(wrAddr) := wrData
}
```

参考文献

[1] Krste Asanović, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, Sagar Karandikar, Ben Keller, Donggyu Kim, John Koenig, Yunsup Lee, Eric Love, Martin Maas, Albert Magyar, Howard Mao, Miquel Moreto, Albert Ou, David A. Patterson, Brian Richards, Colin Schmidt, Stephen Twigg, Huy Vo, and Andrew Waterman. The rocket chip generator. Technical Report UCB/EECS-2016-17, EECS Department, University of California, Berkeley, Apr 2016.

[2] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avizienis, John Wawrzynek, and Krste Asanovic. Chisel: constructing hardware in a Scala embedded language. In Patrick Groeneveld, Donatella Sciuto, and Soha Hassoun, editors, *The 49th Annual Design Automation Conference (DAC 2012)*, pages 1216–1225, San Francisco, CA, USA, June 2012. ACM.

[3] William J. Dally, R. Curtis Harting, and Tor M. Aamodt. *Digital design using VHDL: A systems approach*. Cambridge University Press, 2016.

[4] Schuyler Eldridge, Amos Waterland, Margo Seltzer, and Jonathan Appavooand Ajay Joshi. Towards general-purpose neural network computing. In *2015 International Conference on Parallel Architecture and Compilation (PACT)*, pages 99–112, Oct 2015.

[5] Farzaf Fatollahi-Fard, David Donofrio, George Michelogiannakis, and John Shalf. Opensoc fabric: On-chip network generator. In *2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 194–203, April 2016.

[6] Martin Schoeberl. Lipsi: Probably the smallest processor in the world. In *Architecture of Computing Systems – ARCS 2018*, pages 18–30. Springer International Publishing, 2018.

[7] Martin Schoeberl, David VH Chong, Wolfgang Puffitsch, and Jens Sparsø. A time-predictable memory network-on-chip. In *Proceedings of the 14th International Workshop on Worst-Case Execution Time Analysis (WCET 2014)*, pages 53–62, Madrid, Spain, July 2014.

[8] Martin Schoeberl and Morten Borup Petersen. Leros: The return of the accumulator machine. In Martin Schoeberl, Thilo Pionteck, Sascha Uhrig, Jürgen Brehm, and Christian Hochberger, editors, *Architecture of Computing Systems - ARCS 2019 - 32nd International Conference, Proceedings*, pages 115–127. Springer, 1 2019.

[9] Martin Schoeberl, Luca Pezzarossa, and Jens Sparsø. A minimal network interface for a simple network-on-chip. In Martin Schoeberl, Thilo Pionteck, Sascha Uhrig, Jürgen Brehm, and Christian Hochberger, editors, *Architecture of Computing Systems - ARCS 2019*, pages 295–307. Springer, 1 2019.

[10] Martin Schoeberl, Wolfgang Puffitsch, Stefan Hepp, Benedikt Huber, and Daniel Prokesch. Patmos: A time-predictable microprocessor. *Real-Time Systems*, 54(2):389–423, Apr 2018.

[11] Martin Schoeberl, Tórur Biskopstø Strøm, Oktay Baris, and Jens Sparsø. Scratchpad memories with ownership. In *2019 Design, Automation and Test in Europe Conference Exhibition (DATE)*, 2019.

[12] Bill Venners, Lex Spoon, and Martin Odersky. *Programming in Scala, 3rd Edition.* Artima Inc, 2016.

[13] Andrew Waterman, Yunsup Lee, David A. Patterson, and Krste Asanovic. The risc-v instruction set manual, volume i: Base user-level isa. Technical Report UCB/EECS-2011-62, EECS Department, University of California, Berkeley, May 2011.

[14] Michael Zimmer. *Predictable Processors for Mixed-Criticality Systems and Precision-Timed I/O.* PhD thesis, EECS Department, University of California, Berkeley, Aug 2015.